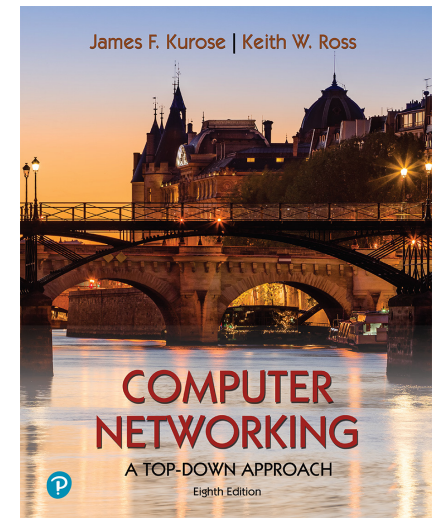


Livello di rete:

Il Piano di Controllo – Control Plane

- Il Routing-



Computer Networking: A Top-Down Approach

8th edition

Jim Kurose, Keith Ross

Pearson, 2020

Network-layer - funzionalità

- **Inoltro (forwarding)**: dirigere i pacchetti dall'ingresso del router, all'uscita appropriata del router *data plane*
- **Instradamento (routing)**: determinare la rotta dei pacchetti dalla sorgente alla destinazione *control plane*
- Nel tragitto da sorgente a destinazione di un datagramma, il ruolo primario del **piano di dati** di ciascun router intermedio è quello di inoltrare il datagramma da link d'ingresso a link d'uscita, mentre quello del **piano di controllo** è coordinare queste azioni di inoltro locali in modo che i datagrammi vengano instradati su percorsi di router tra mittente e destinatario

Network layer: roadmap del piano di controllo

- **Introduzione**
- Protocolli di routing
 - link state
 - distance vector
- Intra-ISP routing: OSPF
- Routing tra ISPs: BGP

Il Piano di Controllo: obiettivi

- Comprendere i principi alla base del piano di controllo a livello di rete:
 - Algoritmi di routing tradizionali
 - Gestione e configurazione della rete
- Come tutto questo viene implementato in internet
 - Open Shortest Path First (OSPF)
 - Border Gateway Protocol (BGP)

Il Piano di Controllo: obiettivi

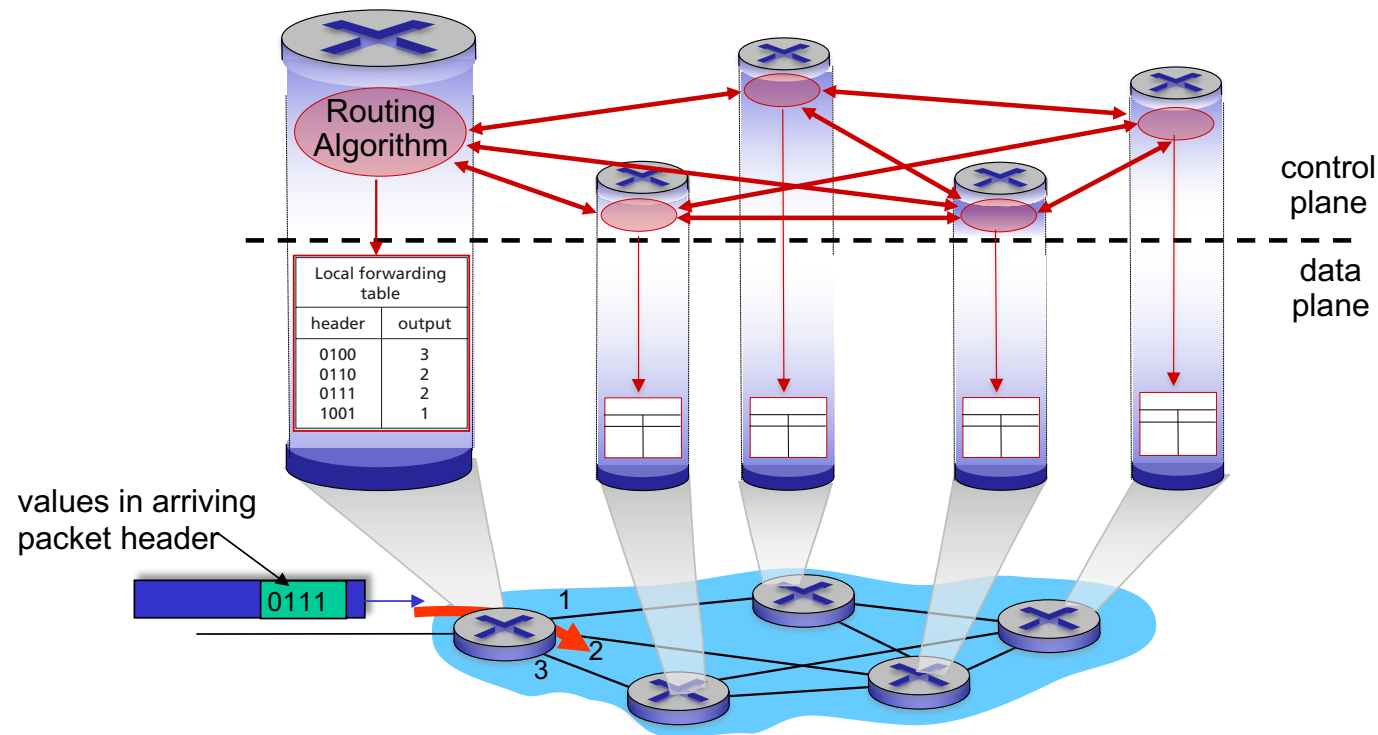
Esistono due approcci per strutturare il piano di controllo della rete

- Per-router control (tradizionale)

-  ■ Logically centralized control (Software Defined Networking, SDN)

Per-router control plane

Le componenti dei singoli algoritmi di routing, in ciascun router, interagiscono nel piano di controllo.



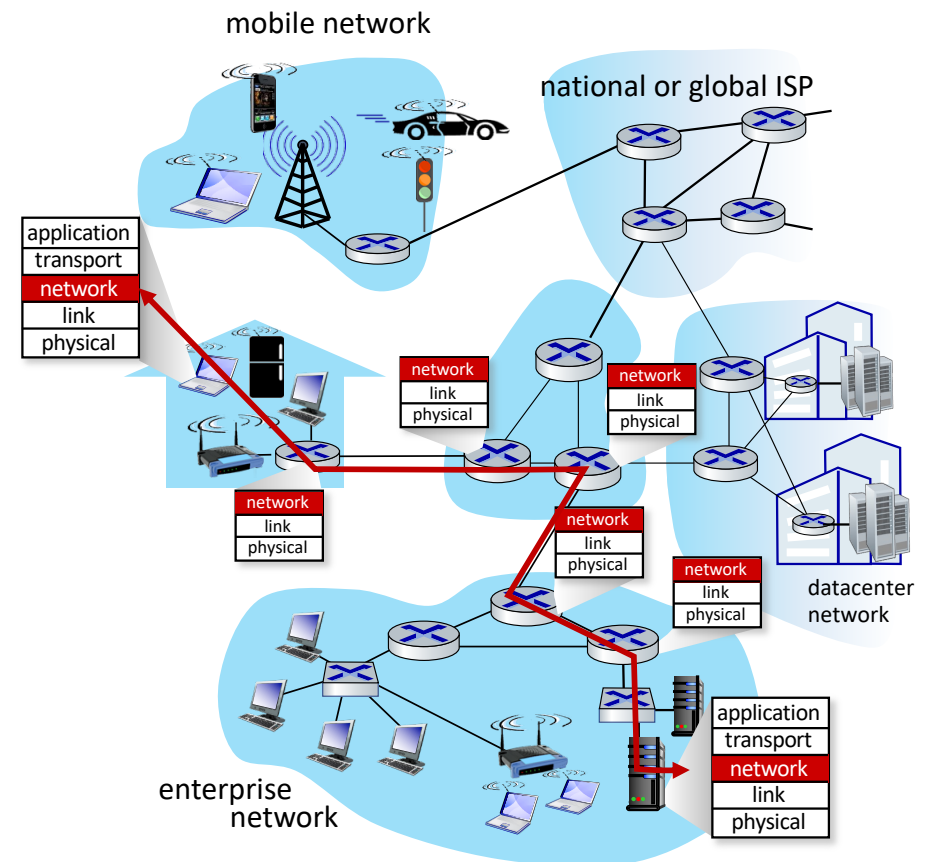
Network layer: roadmap del piano di controllo

- Introduzione
- **Protocolli di Routing**
 - link state
 - distance vector
- Intra-ISP routing: OSPF
- Routing tra ISPs: BGP

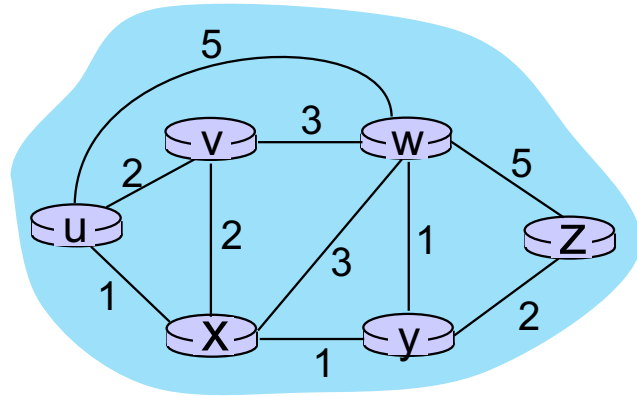
Protocollo di Routing

Obiettivo dei protocolli di routing: determinare percorsi “buoni” (rotte), dall’host mittente all’host destinatario, attraverso i router di rete

- **Percorso:** sequenza di router che i pacchetti devono attraversare dalla sorgente alla destinazione
- **“Buono”:** il meno “costoso”, il più “veloce”, il “meno congestionato”
- Routing: una delle sfide “top-10” in ambito networking!



Costo di un link: astrazione del grafo



graph: $G = (N, E)$

$c_{a,b}$: costo di un link *diretto* che connette a, b

e.g., $c_{w,z} = 5$,

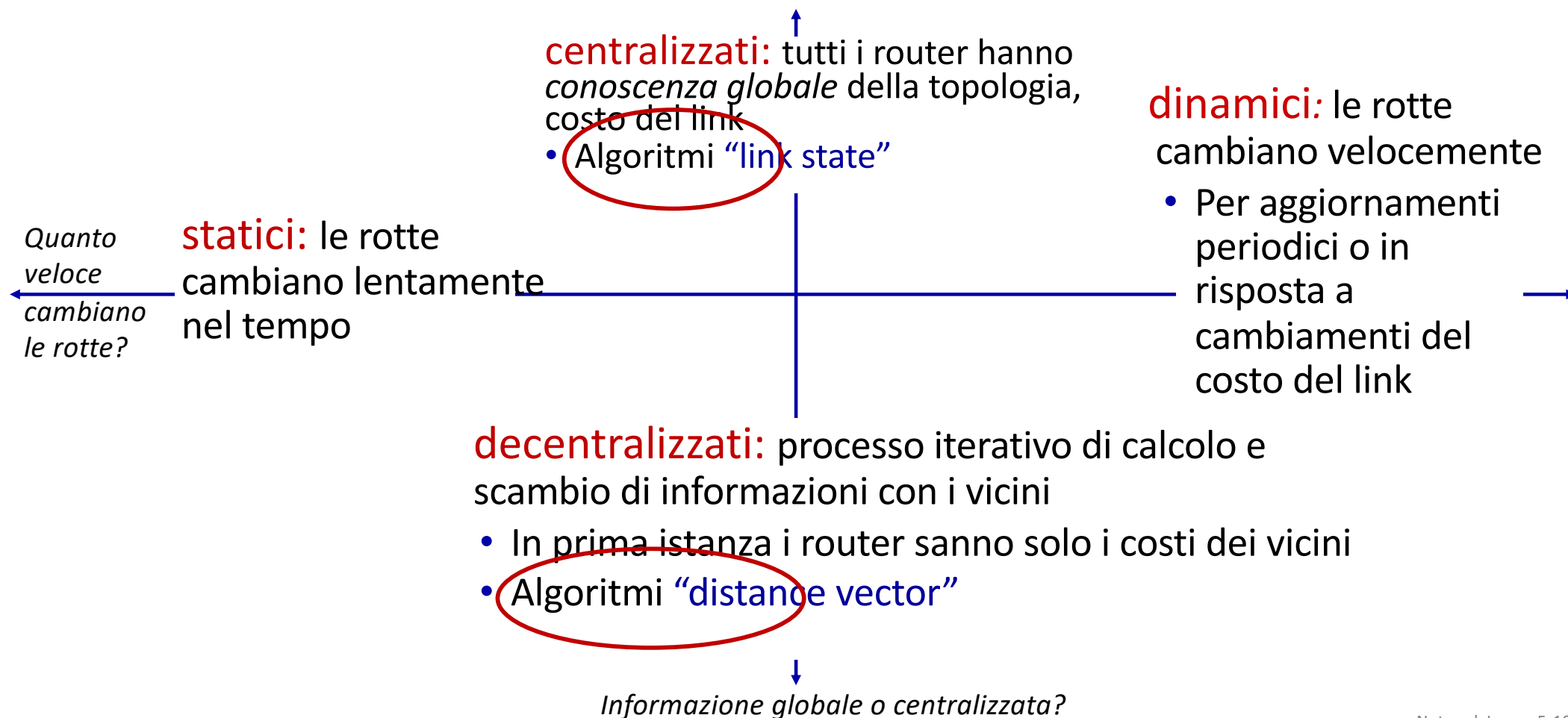
$c_{u,z} = \infty$

Costo definito dall'operatore di rete: può sempre essere 1, o inversamente proporzionale alla banda, o inversamente proporzionale alla congestione.

N : insieme dei (nodi) routers = $\{ u, v, w, x, y, z \}$

E : insieme di (archi) links = $\{ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) \}$

Classificazione algoritmi di routing



Network layer: roadmap del piano di controllo

- Introduzione
- **Protocolli di Routing**
 - link state
 - distance vector
- Intra-ISP routing: OSPF
- Routing tra ISPs: BGP

Algoritmi di routing link-state - Dijkstra

- **centralizzato**: la topologia della rete, i costi dei link sono noti a *tutti* i nodi
 - Invio di messaggi “link state broadcast”
 - Tutti i nodi hanno le stesse informazioni
- Calcola il percorso dal costo minimo da un nodo (“source”) a tutti gli altri
 - Fornisce la *forwarding table* per quel nodo
- **iterativo**: dopo k iterazioni, si conoscono i percorsi a costo minimo per k destinazioni

notazione

- $c_{x,y}$: costo diretto del link dal nodo x al nodo y ; $= \infty$ se non sono vicini diretti
- $D(v)$: stima *corrente* del costo del percorso dal costo minimo dalla sorgente alla destinazione v
- $p(v)$: nodo predecessore lungo il percorso dalla sorgente a v
- N' : insieme di nodi il cui percorso dal costo minimo è noto *definitivamente*

Algoritmi di routing link-state - Dijkstra

1 *Inizializzazione:*

2 $N' = \{u\}$ /* compute least-cost-path from u to all other nodes */

3 for all nodes v

4 if v adjacent to u /* u initially knows direct-path-cost only to direct neighbors */

5 then $D(v) = c_{u,v}$ /* but may not be *minimum* cost! */

6 else $D(v) = \infty$

7

8 *Loop*

9 find w not in N' such that $D(w)$ is a minimum

10 add w to N'

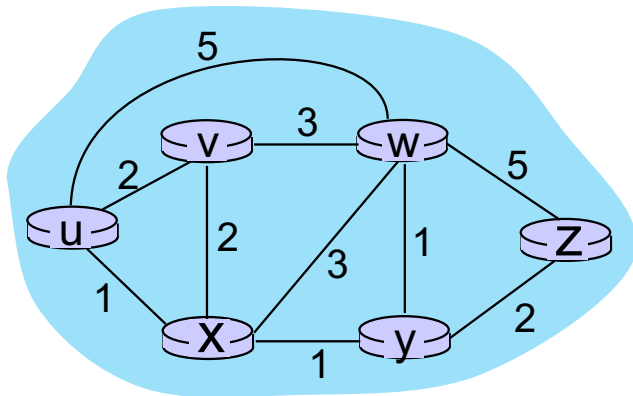
11 update $D(v)$ for all v adjacent to w and not in N' :

12 $D(v) = \min (D(v), D(w) + c_{w,v})$ /* new least-path-cost to v is either old
13 least-cost-path to v or known least-cost-path
14 to w plus direct-cost from w to v */

15 *until all nodes in N'*

Dijkstra's algorithm: un esempio

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1						
2						
3						
4						
5						

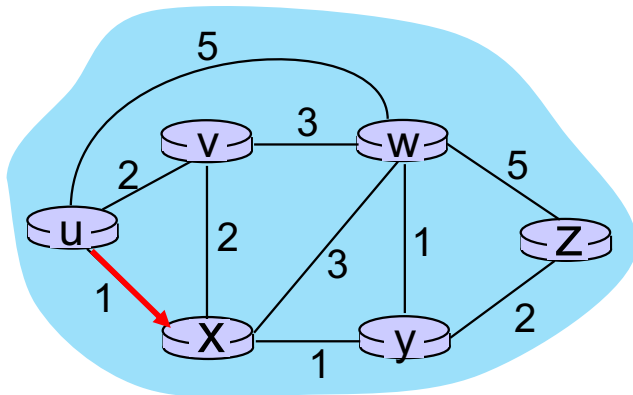


Initialization (step 0):

For all a : if a adjacent to u then $D(a) = c_{u,a}$

Dijkstra's algorithm: un esempio

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	u, x					
2						
3						
4						
5						

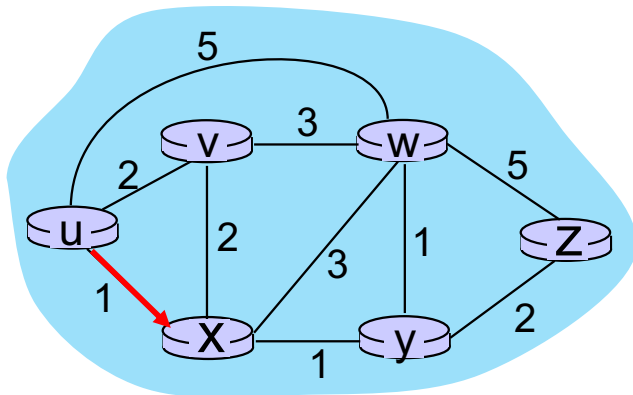


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: un esempio

Step	N'	v D(v),p(v)	w D(w),p(w)	x D(x),p(x)	y D(y),p(y)	z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2						
3						
4						
5						



8 Loop

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

$$D(v) = \min (D(v), D(x) + c_{x,v}) = \min(2, 1+2) = 2$$

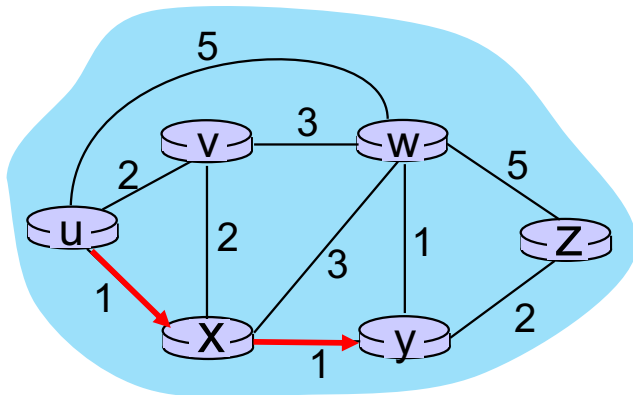
$$D(w) = \min (D(w), D(x) + c_{x,w}) = \min(5, 1+3) = 4$$

$$D(y) = \min (D(y), D(x) + c_{x,y}) = \min(\infty, 1+1) = 2$$

NEW!

Dijkstra's algorithm: un esempio

Step	N'	v D(v),p(v)	w D(w),p(w)	x D(x),p(x)	y D(y),p(y)	z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy					
3						
4						
5						

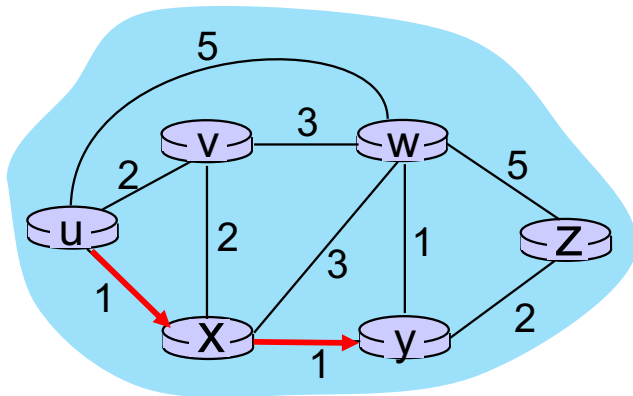


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: un esempio

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3						
4						
5						



8 Loop

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

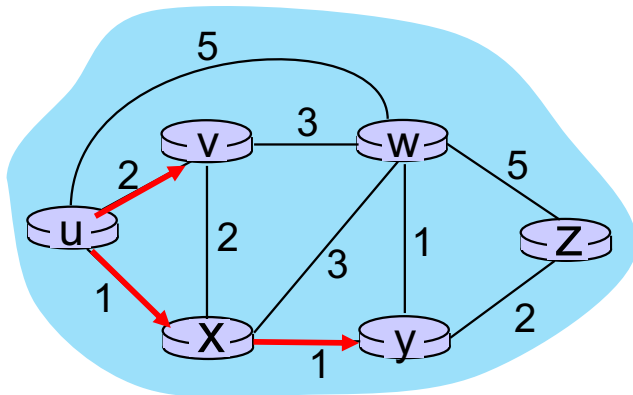
$$D(w) = \min (D(w), D(y) + c_{y,w}) = \min (4, 2+1) = 3$$

$$D(z) = \min (D(z), D(y) + c_{y,z}) = \min (\infty, 2+2) = 4$$



Dijkstra's algorithm: un esempio

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv					
4						
5						

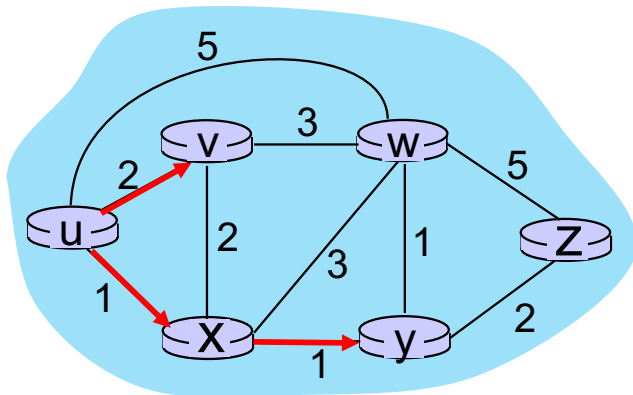


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: un esempio

Step	N'	v D(v),p(v)	w D(w),p(w)	x D(x),p(x)	y D(y),p(y)	z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4						
5						



8 *Loop*

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

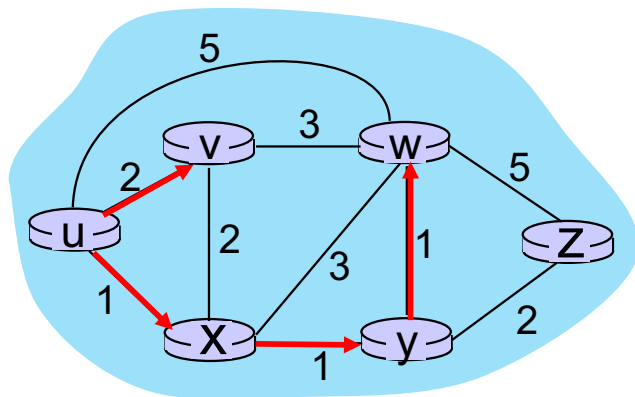
11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

$$D(w) = \min (D(w), D(v) + c_{v,w}) = \min (3, 2+3) = 3$$

Dijkstra's algorithm: un esempio

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					
5						

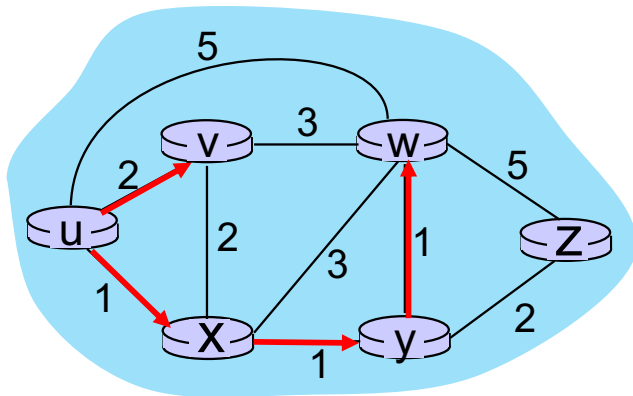


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: un esempio

Step	N'	v D(v),p(v)	w D(w),p(w)	x D(x),p(x)	y D(y),p(y)	z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5						



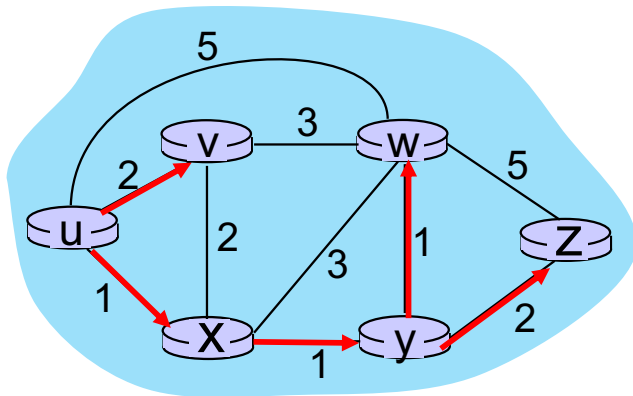
8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :
 $D(b) = \min (D(b), D(a) + c_{a,b})$

$$D(z) = \min (D(z), D(w) + c_{w,z}) = \min (4, 3+5) = 4$$

Dijkstra's algorithm: un esempio

Step	N'	v D(v),p(v)	w D(w),p(w)	x D(x),p(x)	y D(y),p(y)	z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					

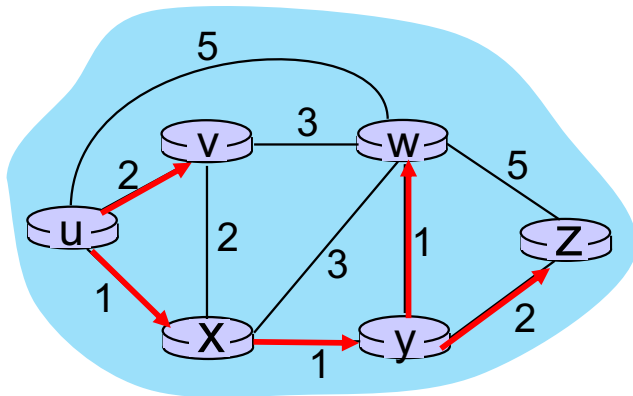


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: un esempio

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					



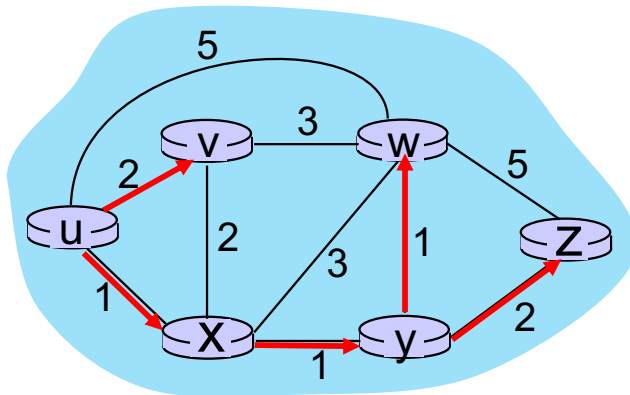
8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

Dijkstra's algorithm: un esempio

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	u, x	2, u	4, x		2, x	∞
2	u, x, y	2, u	3, y			4, y
3	u, x, y, v		3, y			4, y
4	u, x, y, v, w					4, y
5	u, x, y, v, w, z					



Initialization (step 0): For all a : if a adjacent to then $D(a) = c_{u,a}$

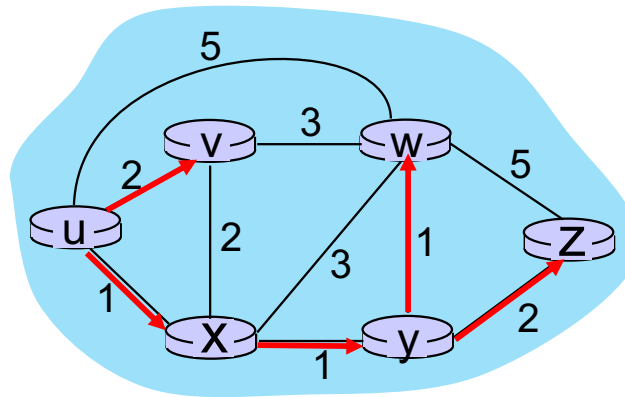
find a not in N' such that $D(a)$ is a minimum

add a to N'

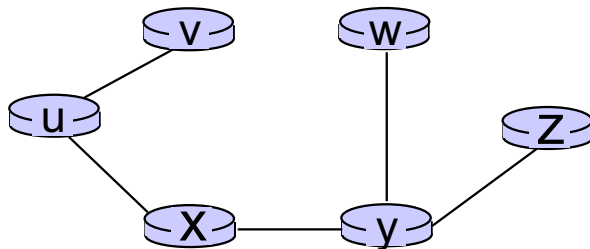
update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

Dijkstra's algorithm: un esempio



resulting least-cost-path tree from u:



resulting forwarding table in u:

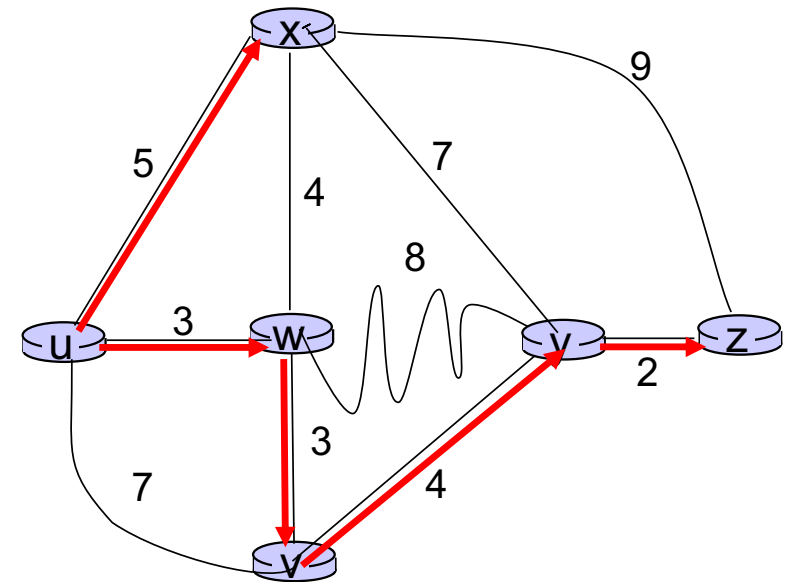
destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

route from u to v directly

route from u to all other destinations via x

Dijkstra's algorithm: un altro esempio

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwX	6,w			11,w	14,x
3	uwXv				10,v	14,x
4	uwXvy					12,y
5	uwXvyZ					



notes:

- Costruire l'albero del least-cost-path tracciando i nodi predecessori
- Possono esistere dei pareggi, che possono essere risolti in modo arbitrario

Algoritmo di Dijkstra: discussione

Complessità algoritmo: n nodi

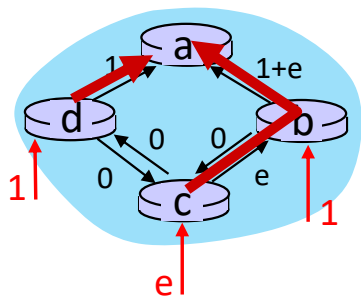
- Ognuna delle n iterazioni: deve passare per tutti i nodi, w , non in N
- $n(n+1)/2$ confronti: complessità $O(n^2)$
- La più efficiente implementazione, sfruttando lo heap: $O(n \log n)$

Complessità messaggi:

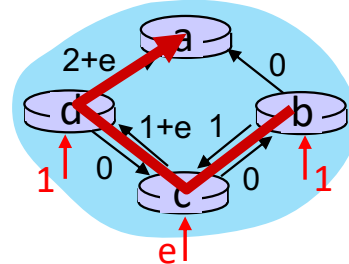
- Ogni router manda in *broadcast* le sue informazioni “link state” agli altri n routers
- Algoritmi di broadcast efficienti (e interessanti!): $O(n)$ attraversamenti di link per diffondere un messaggio di broadcast da una sorgente
- Il messaggio di ciascun router attraversa $O(n)$ link: complessità complessiva del messaggio: $O(n^2)$

Dijkstra's algorithm: oscillations possible

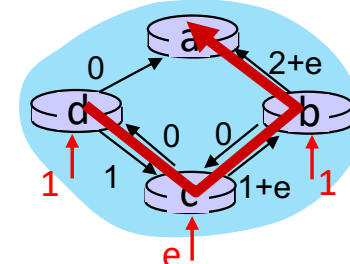
- Quando i costi dei link dipendono dal volume del traffico, avviene un' **oscillazione delle rotte**
- Esempio:
 - routing to destination a, traffic entering at d, c, e with rates 1, e (<1), 1
 - link costs are directional, and volume-dependent



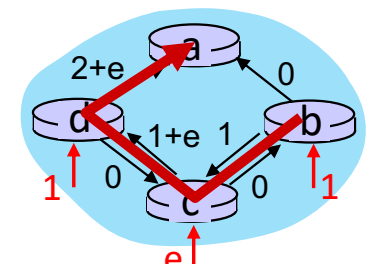
inizio



Dati questi costi,
Trovare un nuovo routing....
Risulta in nuovi costi



Dati questi costi,
Trovare un nuovo routing....
Risulta in nuovi costi



Dati questi costi,
Trovare un nuovo routing....
Risulta in nuovi costi

Network layer: roadmap del piano di controllo

- Introduzione
- **Protocolli di Routing**
 - link state
 - **distance vector**
- Intra-ISP routing: OSPF
- Routing tra ISPs: BGP

Algoritmi Distance Vector

Basati sull'equazione di *Bellman-Ford* (BF) (dynamic programming):

Equazione di Bellman-Ford

$D_x(y)$: costo del least-cost path da x a y .

allora:

$$D_x(y) = \min_v \{ c_{x,v} + D_v(y) \}$$

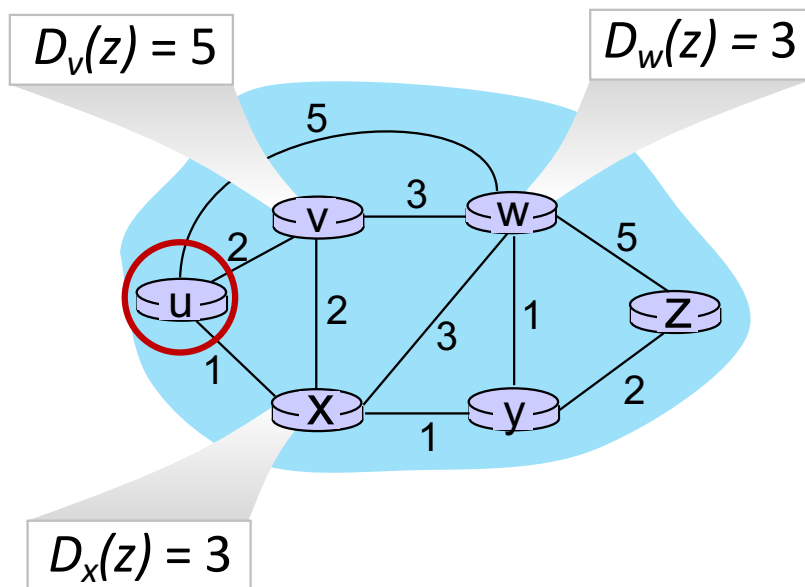
$\min$ preso da tutti i vicini v di x

Costo diretto del link da x a v

stima del least-cost-path da v a y

Bellman-Ford: un esempio

Se i nodi vicini a u , che sono x, v, w , sanno che per z :



Equazione di Bellman-Ford dice:

$$\begin{aligned} D_u(z) &= \min \{ c_{u,v} + D_v(z), \\ &\quad c_{u,x} + D_x(z), \\ &\quad c_{u,w} + D_w(z) \} \\ &= \min \{ 2 + 5, \\ &\quad 1 + 3, \\ &\quad 5 + 3 \} = 4 \end{aligned}$$

Il nodo (x) è il next hop sul least-cost path stimato per destinazione (z)

Algoritmi Distance Vector

idea:

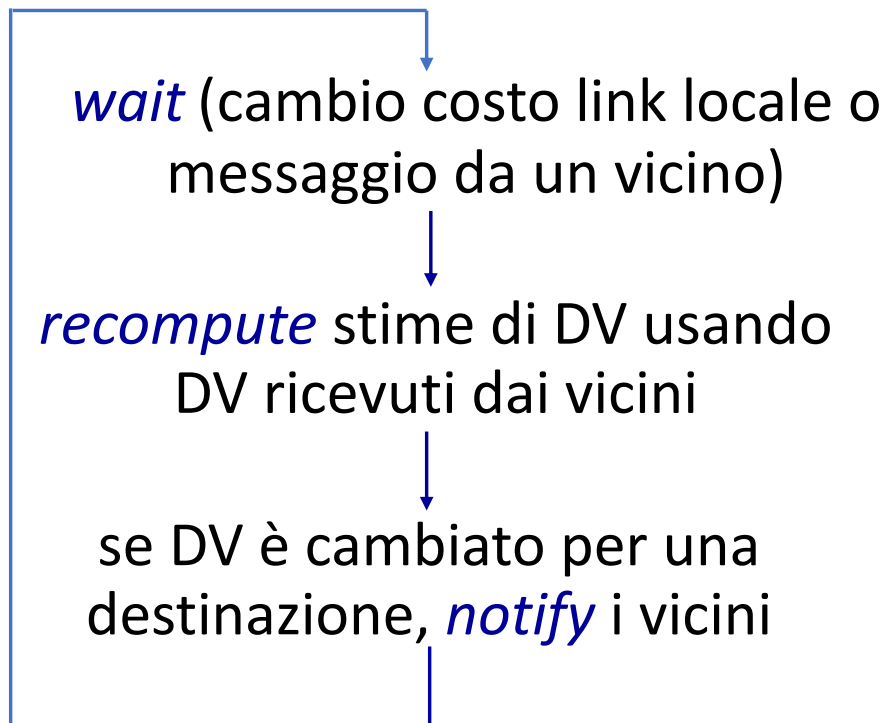
- Di tanto in tanto, ciascun nodo invia la propria stima del vettore delle distanze (DV) ai vicini
- Quando x riceve nuove stime per il DV da qualsiasi vicino, aggiorna il suo DV usando l'equazione B-F:

$$D_x(y) \leftarrow \min_v \{c_{x,v} + D_v(y)\} \text{ per ogni nodo } y \in N$$

- In condizioni normali, la stima $D_x(y)$ converge all'effettivo least cost $d_x(y)$

Algoritmi Distance Vector

ogni nodo:



iterativo, asincrono: ogni iterazione locale è causata da:

- Cambio costo link locale
- DV update da un vicino

distribuito, self-stopping: ogni nodo notifica i vicini solo quando il suo DV cambia

- I vicini poi notificano i loro vicini— *solo se necessario*
- Nessuna notifica ricevuta, nessuna azione!

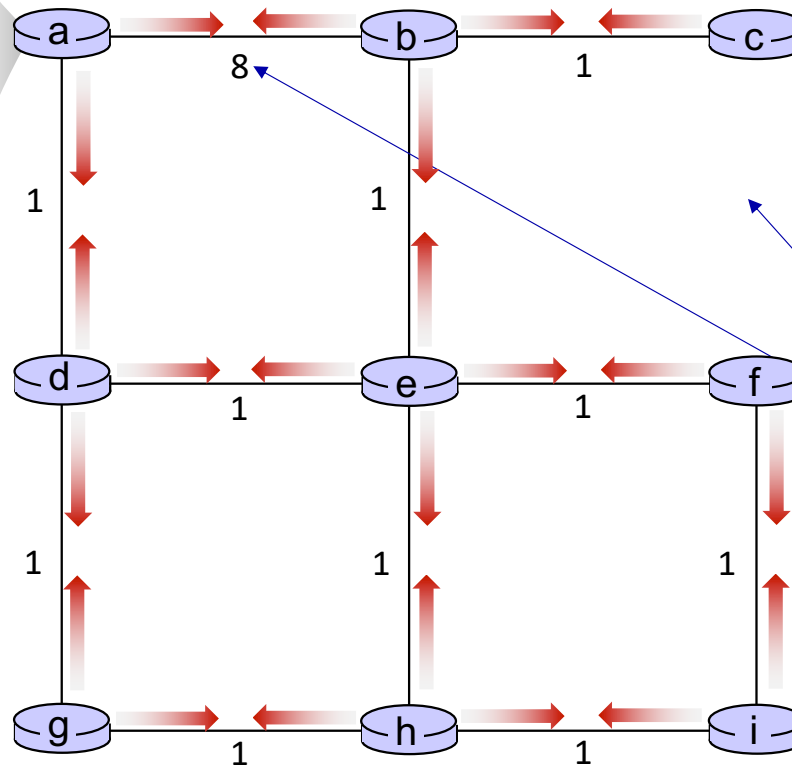
Algoritmi Distance Vector: un esempio



t=0

- Tutti i nodi hanno le stime dei DV SOLO per i nodo direttamente collegato
- Tutti i nodi mandano i loro DV locali ai vicini

DV di a:	
$D_a(a)$	0
$D_a(b)$	8
$D_a(c)$	∞
$D_a(d)$	1
$D_a(e)$	∞
$D_a(f)$	∞
$D_a(g)$	∞
$D_a(h)$	∞
$D_a(i)$	∞



Qualche asimmetria:

- Link mancante
- Costo più alto

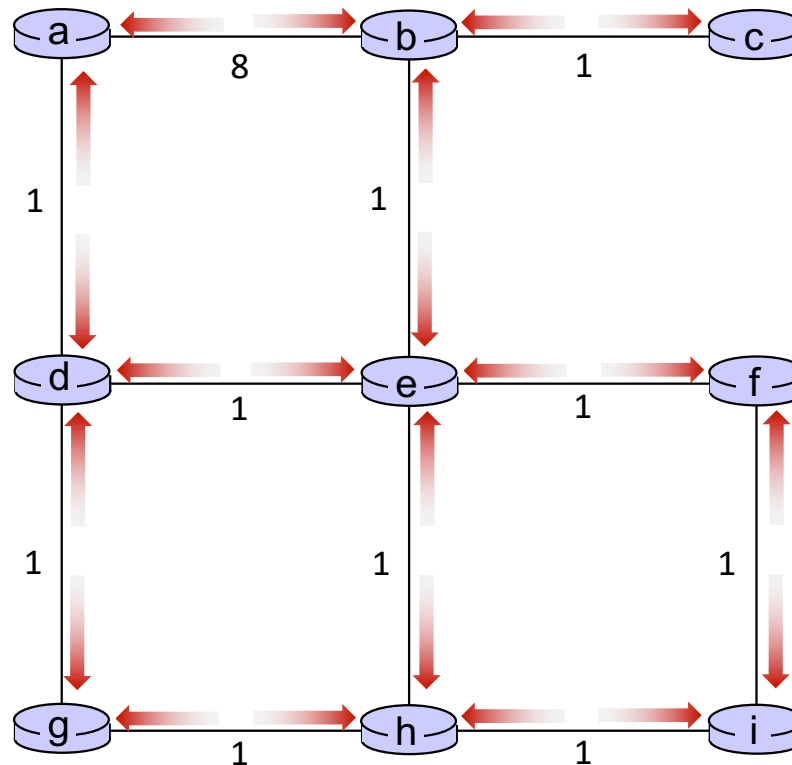
Algoritmi Distance Vector: iterazione



$t=1$

Tutti i nodi:

- Ricevono DV dai vicini
- Calcolano il loro nuovo DV
- Mandano il loro nuovo DV ai nodi vicini



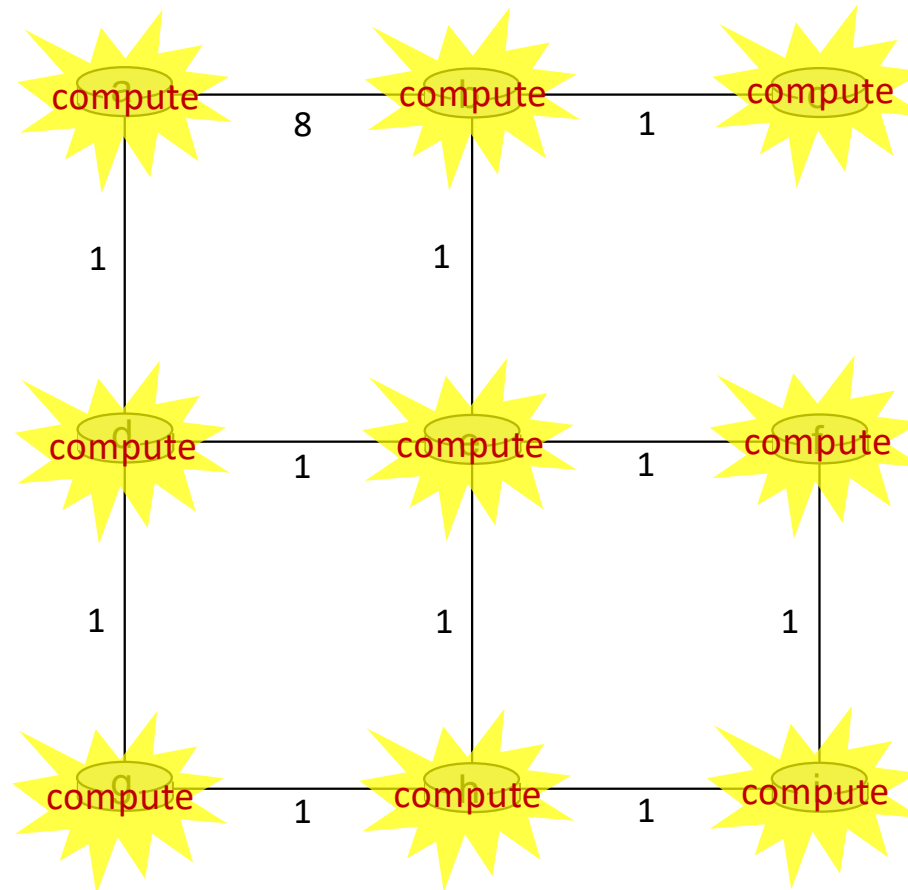
Algoritmi Distance Vector: iterazione



$t=1$

Tutti i nodi:

- Ricevono DV dai vicini
- Calcolano il loro nuovo DV
- Mandano il loro nuovo DV ai nodi vicini

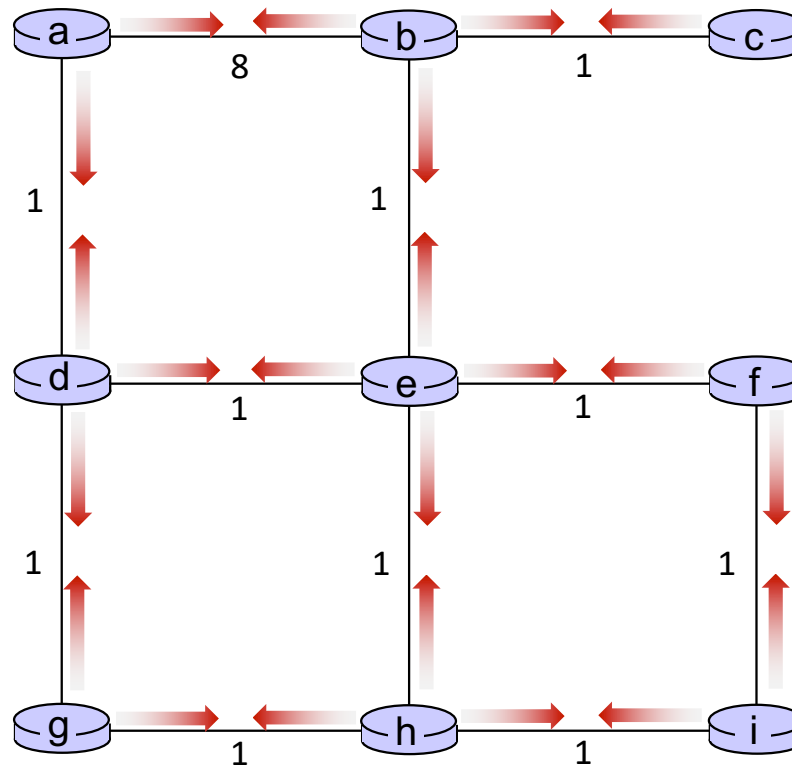


Algoritmi Distance Vector: iterazione



t=1

- Tutti i nodi:
- Ricevono DV dai vicini
- Calcolano il loro nuovo DV
- Mandano il loro nuovo DV ai nodi vicini



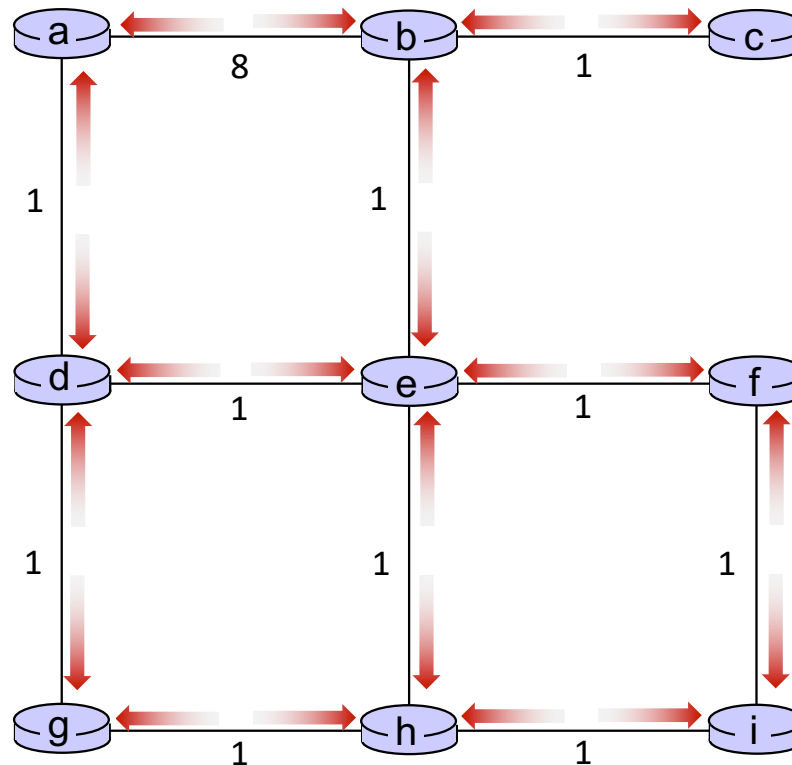
Algoritmi Distance Vector: iterazione



t=2

Tutti i nodi:

- Ricevono i DV dai vicini
- Calcolano il loro nuovo DV
- Mandano il loro nuovo DV ai nodi vicini



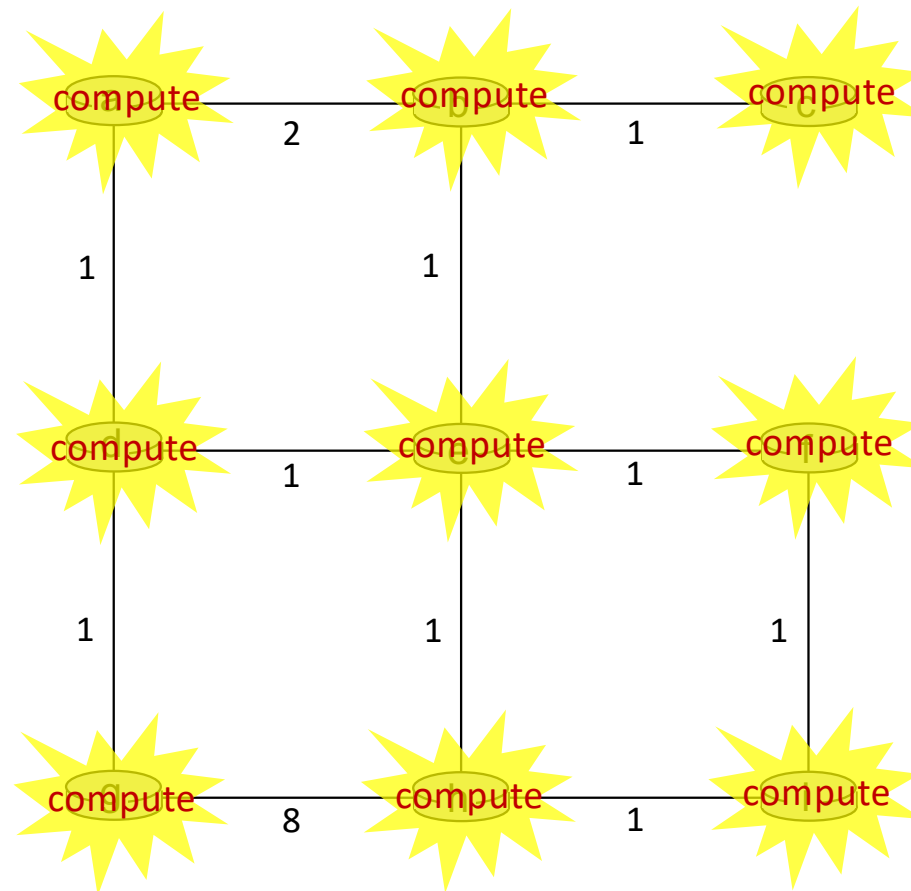
Algoritmi Distance Vector: iterazione



t=2

Tutti i nodi:

- Ricevono i DV dai vicini
- Calcolano il loro nuovo DV
- Mandano il loro nuovo DV ai nodi vicini



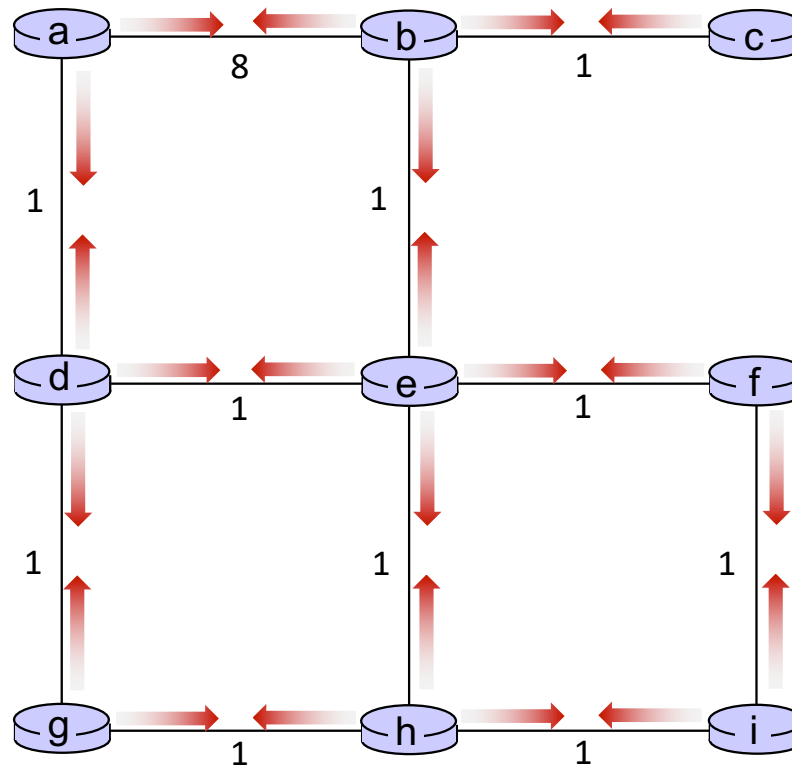
Algoritmi Distance Vector: iterazione



t=2

Tutti i nodi:

- Ricevono i DV dai vicini
- Calcolano il loro nuovo DV
- Mandano il loro nuovo DV ai nodi vicini



Algoritmi Distance Vector: iterazione

.... E così via

Esaminiamo ora i calcoli iterative dei nodi

Distance Vector: esempio



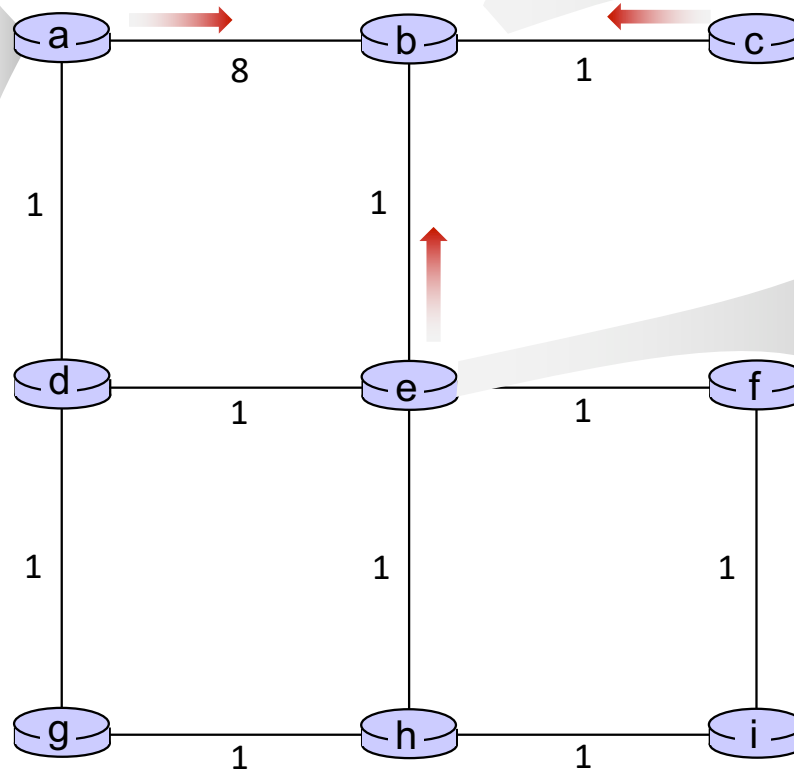
t=1

- b riceve DVs da a, c, e

DV di a:
$D_a(a)=0$
$D_a(b)=8$
$D_a(c)=\infty$
$D_a(d)=1$
$D_a(e)=\infty$
$D_a(f)=\infty$
$D_a(g)=\infty$
$D_a(h)=\infty$
$D_a(i)=\infty$

DV di b:	
$D_b(a) = 8$	$D_b(f) = \infty$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = \infty$	$D_b(h) = \infty$
$D_b(e) = 1$	$D_b(i) = \infty$

DV di c:
$D_c(a)=\infty$
$D_c(b)=1$
$D_c(c)=0$
$D_c(d)=\infty$
$D_c(e)=\infty$
$D_c(f)=\infty$
$D_c(g)=\infty$
$D_c(h)=\infty$
$D_c(i)=\infty$



DV di e:
$D_e(a)=\infty$
$D_e(b)=1$
$D_e(c)=\infty$
$D_e(d)=1$
$D_e(e)=0$
$D_e(f)=1$
$D_e(g)=\infty$
$D_e(h)=1$
$D_e(i)=\infty$

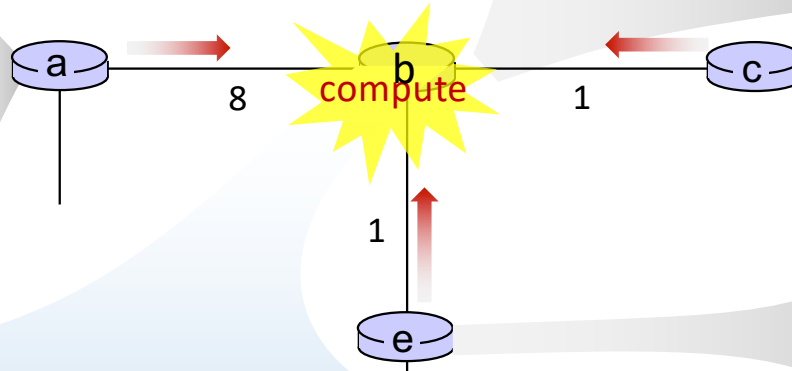
Distance Vector: esempio



t=1

- b riceve DVs da a, c, e, calcola:

DV in a:
$D_a(a)=0$
$D_a(b)=8$
$D_a(c)=\infty$
$D_a(d)=1$
$D_a(e)=\infty$
$D_a(f)=\infty$
$D_a(g)=\infty$
$D_a(h)=\infty$
$D_a(i)=\infty$



DV in b:	
$D_b(a) = 8$	$D_b(f) = \infty$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = \infty$	$D_b(h) = \infty$
$D_b(e) = 1$	$D_b(i) = \infty$

DV in c:
$D_c(a)=\infty$
$D_c(b)=1$
$D_c(c)=0$
$D_c(d)=\infty$
$D_c(e)=\infty$
$D_c(f)=\infty$
$D_c(g)=\infty$
$D_c(h)=\infty$
$D_c(i)=\infty$

$$D_b(a) = \min\{c_{b,a} + D_a(a), c_{b,c} + D_c(a), c_{b,e} + D_e(a)\} = \min\{8, \infty, \infty\} = 8$$

$$D_b(c) = \min\{c_{b,a} + D_a(c), c_{b,c} + D_c(c), c_{b,e} + D_e(c)\} = \min\{\infty, 1, \infty\} = 1$$

$$D_b(d) = \min\{c_{b,a} + D_a(d), c_{b,c} + D_c(d), c_{b,e} + D_e(d)\} = \min\{9, \infty, 2\} = 2$$

$$D_b(e) = \min\{c_{b,a} + D_a(e), c_{b,c} + D_c(e), c_{b,e} + D_e(e)\} = \min\{\infty, \infty, 1\} = 1$$

$$D_b(f) = \min\{c_{b,a} + D_a(f), c_{b,c} + D_c(f), c_{b,e} + D_e(f)\} = \min\{\infty, \infty, 2\} = 2$$

$$D_b(g) = \min\{c_{b,a} + D_a(g), c_{b,c} + D_c(g), c_{b,e} + D_e(g)\} = \min\{\infty, \infty, \infty\} = \infty$$

$$D_b(h) = \min\{c_{b,a} + D_a(h), c_{b,c} + D_c(h), c_{b,e} + D_e(h)\} = \min\{\infty, \infty, 2\} = 2$$

$$D_b(i) = \min\{c_{b,a} + D_a(i), c_{b,c} + D_c(i), c_{b,e} + D_e(i)\} = \min\{\infty, \infty, \infty\} = \infty$$

DV in b:	
$D_b(a) = 8$	$D_b(f) = 2$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = 2$	$D_b(h) = 2$
$D_b(e) = 1$	$D_b(i) = \infty$

DV in e:
$D_e(a)=\infty$
$D_e(b)=1$
$D_e(c)=\infty$
$D_e(d)=1$
$D_e(e)=0$
$D_e(f)=1$
$D_e(g)=\infty$
$D_e(h)=1$
$D_e(i)=\infty$

Distance Vector: esempio



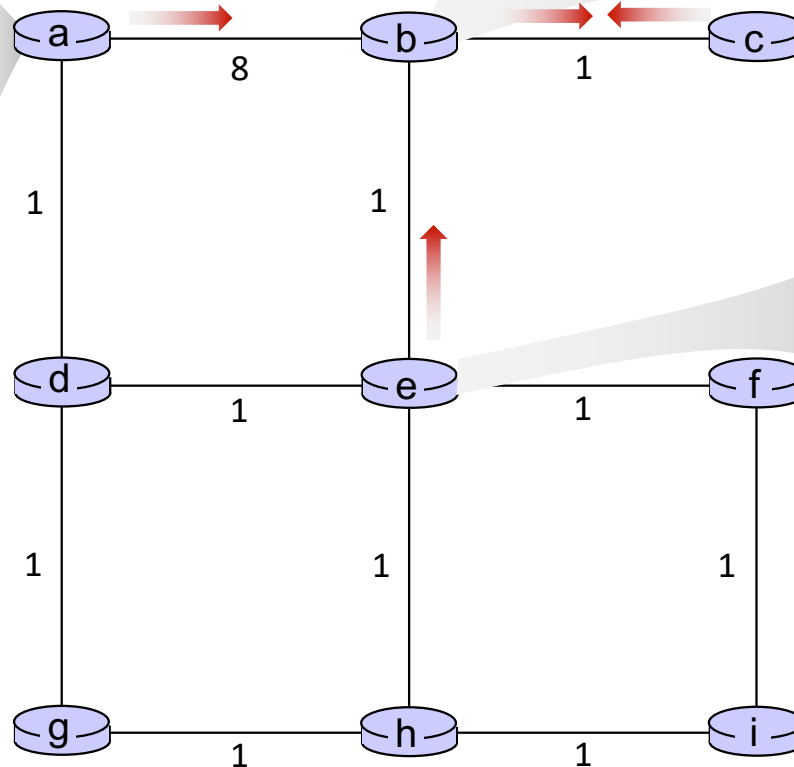
t=1

- c riceve DVs da b

DV in a:
$D_a(a)=0$
$D_a(b)=8$
$D_a(c)=\infty$
$D_a(d)=1$
$D_a(e)=\infty$
$D_a(f)=\infty$
$D_a(g)=\infty$
$D_a(h)=\infty$
$D_a(i)=\infty$

DV in b:	
$D_b(a) = 8$	$D_b(f) = \infty$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = \infty$	$D_b(h) = \infty$
$D_b(e) = 1$	$D_b(i) = \infty$

DV in c:
$D_c(a)=\infty$
$D_c(b)=1$
$D_c(c)=0$
$D_c(d)=\infty$
$D_c(e)=\infty$
$D_c(f)=\infty$
$D_c(g)=\infty$
$D_c(h)=\infty$
$D_c(i)=\infty$



DV in e:
$D_e(a)=\infty$
$D_e(b)=1$
$D_e(c)=\infty$
$D_e(d)=1$
$D_e(e)=0$
$D_e(f)=1$
$D_e(g)=\infty$
$D_e(h)=1$
$D_e(i)=\infty$

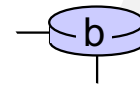
Distance Vector: esempio



$t=1$

- c riceve DVs da b e calcola:

$$\begin{aligned} D_c(a) &= \min\{c_{c,b} + D_b(a)\} = 1 + 8 = 9 \\ D_c(b) &= \min\{c_{c,b} + D_b(b)\} = 1 + 0 = 1 \\ D_c(d) &= \min\{c_{c,b} + D_b(d)\} = 1 + \infty = \infty \\ D_c(e) &= \min\{c_{c,b} + D_b(e)\} = 1 + 1 = 2 \\ D_c(f) &= \min\{c_{c,b} + D_b(f)\} = 1 + \infty = \infty \\ D_c(g) &= \min\{c_{c,b} + D_b(g)\} = 1 + \infty = \infty \\ D_c(h) &= \min\{c_{c,b} + D_b(h)\} = 1 + \infty = \infty \\ D_c(i) &= \min\{c_{c,b} + D_b(i)\} = 1 + \infty = \infty \end{aligned}$$



1

compute

DV di b:

$D_b(a) = 8$	$D_b(f) = \infty$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = \infty$	$D_b(h) = \infty$
$D_b(e) = 1$	$D_b(i) = \infty$

DV di c:

$D_c(a) = \infty$
$D_c(b) = 1$
$D_c(c) = 0$
$D_c(d) = \infty$
$D_c(e) = \infty$
$D_c(f) = \infty$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = \infty$

DV di c:

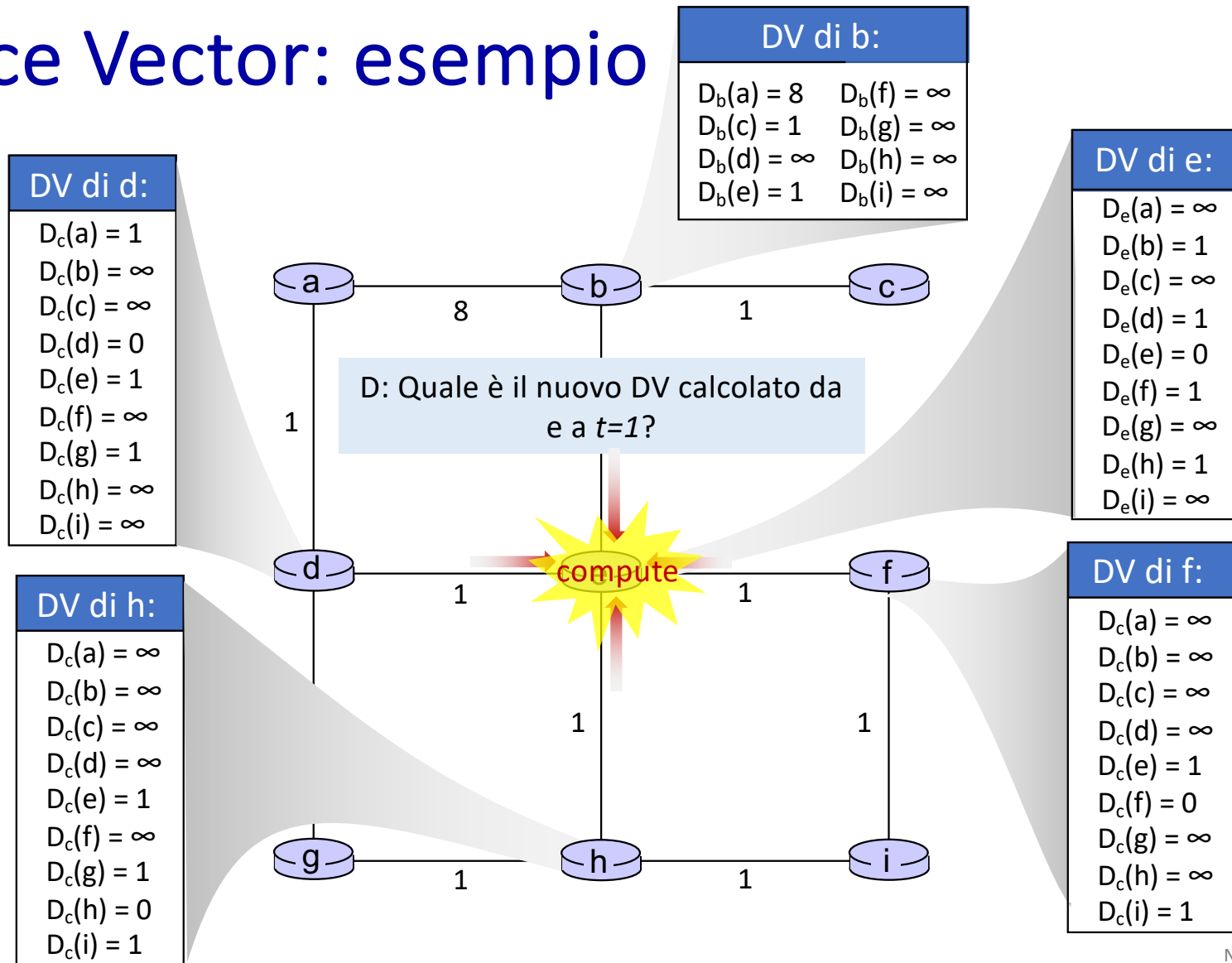
$D_c(a) = 9$
$D_c(b) = 1$
$D_c(c) = 0$
$D_c(d) = 2$
$D_c(e) = \infty$
$D_c(f) = \infty$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = \infty$

Distance Vector: esempio



t=1

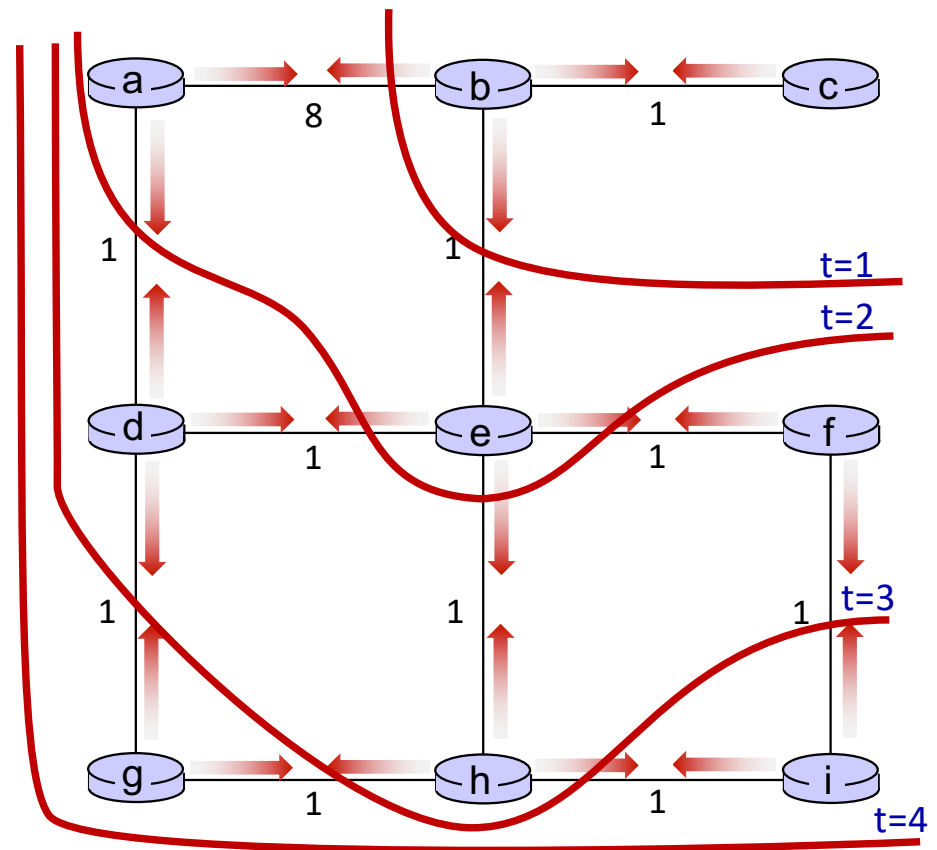
- e riceve DVs da b, d, f, h



Distance Vector: diffusione informazioni di stato

Comunicazione iterativa, dopo il calcolo le informazioni vengono diffuse nella rete:

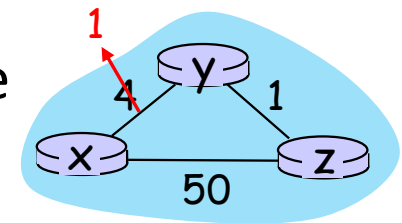
-  **t=0** Lo stato di **c** a t=0 lo conosce solo **c**
-  **t=1** Lo stato di **c** a t=0 è stato propagato a **b**, e può influenzare il calcolo dei DV al massimo fino a **1** hop away, i.e., fino **b**
-  **t=2** Lo stato di **c** a t=0 può ora influenzare il calcolo dei DV fino a **2** hops away, i.e., fino **b** e ora anche ad **a, e**
-  **t=3** Lo stato di **c** a t=0 può influenzare il calcolo dei DV fino a **3** hops away, i.e., fino **d, f, h**
-  **t=4** Lo stato di **c** a t=0 può influenzare il calcolo dei DV fino a **4** hops away, i.e., fino **g, i**



Distance Vector: cambi dei costi nei link

Cambio del costo di un link:

- Il nodo rileva un cambiamento di costo in un suo link locale
- Aggiorna le informazioni di routing, ricalcola il suo DV locale
- Se il DV locale cambia, lo notifica ai vicini



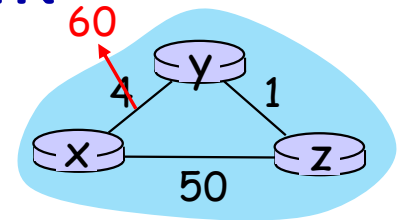
t_0 : y rileva cambio costo link, aggiorna il suo DV, informa i vicini.

t_1 : z e x ricevono aggiornamenti da y, aggiornano il loro DV, calcolano il nuovo percorso a costo minimo per x, z, mandano ai vicini il loro DV.

t_2 : y riceve aggiornamenti da z e x, aggiorna il suo DV. Il costo minimo di y *non cambia*, e y *non* manda altri messaggi.

“le buone
notizie
viaggiano
veloce”

Distance Vector: cambi dei costi nei link



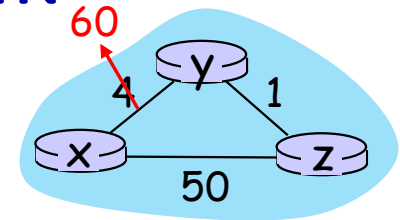
Cambio del costo di un link:

- Il nodo rileva un cambiamento di costo in un suo link locale
- “le cattive notizie viaggiano lente” – count-to-infinity problem:
 - y vede che il nuovo costo per x è 60, ma z ha detto di avere un percorso di costo 5 per x. y calcola “il mio nuovo costo per x sarà 6, via z); notifica z del nuovo costo 6 per x.
 - z sa che la via per x attraverso y ha un nuovo costo 6. z calcola “il mio nuovo costo per x sarà 7, via y), notifica y del nuovo costo 7 per x.
 - y ora sa che il percorso per x via z ha un nuovo costo di 7. y calcola “il mio nuovo costo per x sarà 8 via y), notifica z del nuovo costo 8 per x.
 - z sa che il percorso per x via y ha un nuovo costo 8. z calcola “il mio nuovo costo per x sarà 9 via y), notifica y del nuovo costo 9 per x.
 - ...
- Per risolvere il problema: INVERSIONE AVVELENATA

Distance Vector: cambi dei costi nei link

Inversione Avvelenata

- Se z instrada tramite y per giungere a x
 - z avvertirà y che la sua distanza verso x è infinita
 - $D_z(x) = \infty$ anche se z sa che in realtà $D_z(x) = 5$
 - Finchè z arriverà ad x passando per y, continuerà ad annunciare questo costo
 - y pensa quindi che z non abbia un percorso verso x, e non verrà creato il ciclo, e i costi verranno comunicati correttamente
- NB: Questa tecnica funziona solo per risolvere cicli che si formano tra nodi adiacenti



Confronto tra algoritmi LS e DV

Complessità dei messaggi

LS: n routers, $O(n^2)$ messaggi inviati

DV: scambi tra vicini; il tempo di convergenza varia

Velocità di convergenza

LS: Algoritmo $O(n^2)$, $O(n^2)$ messaggi

- Ci possono essere oscillazioni

DV: tempo di convergenza varia

- Ci possono essere dei cicli
- Problema del count-to-infinity

Robustezza: cosa succede se un router si rompe o viene compromesso?

LS:

- router possono comunicare costi di *link* non corretti
- Ogni router costruisce solo la sua tabella

DV:

- DV router possono comunicare costi di *percorsi non corretti* (“Ho un percorso low cost per ovunque”): *black-holing*
- Ogni DV di ogni router è usato anche dagli altri: l'errore si propaga nella rete
 - <https://arstechnica.com/uncategorized/2008/02/insecure-routing-redirects-youtube-to-pakistan/>

Network layer: roadmap del piano di controllo

- Introduzione
- Protocolli di Routing
 - link state
 - distance vector
- **Intra-ISP routing: OSPF**
- Routing tra ISPs: BGP

Come ottenere un routing scalabile?

La trattazione sul routing fino ad ora è stata puramente teorica

- Tutti i router sono uguali
- La rete è piatta

... questo non è vero in pratica

Scalabilità: miliardi di destinazioni

- Non si possono salvare tutte in tabelle di routing!
- Lo scambio di tabelle di instradamento ingolferebbe i collegamenti!

Autonomia amministrativa:

- Internet: una rete di reti
- Ogni amministratore di rete vuole controllare il routing all'interno della sua rete

L'approccio in Internet per routing scalabile

Aggregare i router in regioni chiamate “Autonomous Systems” (AS) (a.k.a. “domains”)

intra-AS (aka “intra-domain”):
routing tra router *nello stesso AS*
(“*network*”)

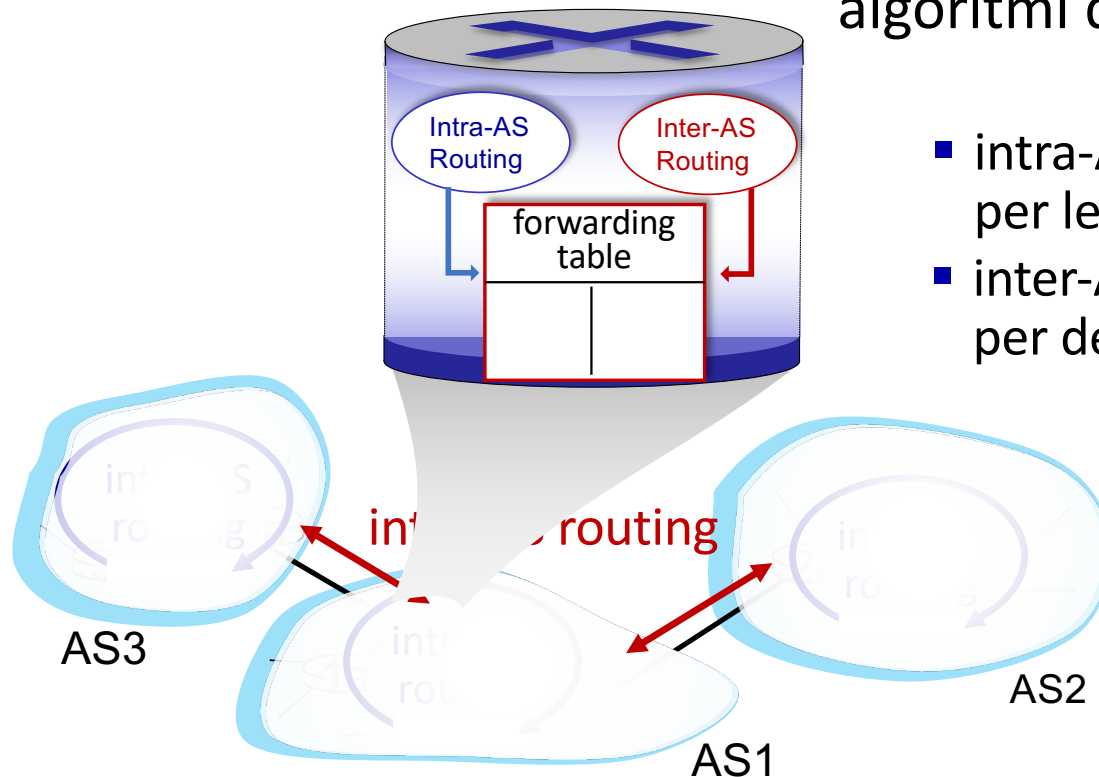
- Tutti i router in un AS devono eseguire lo stesso protocollo di routing intra-domain
- Router in AS diversi possono eseguire protocolli di routing intra-domain diversi
- **Gateway router:** all’ “edge” del suo AS, ha link(s) verso i router(s) in altri AS

inter-AS (aka “inter-domain”):
routing *tra* AS

- gateways eseguono inter-domain routing (oltre che intra-domain routing)

AS interconnessi

La tabella di inoltro è configurata dagli algoritmi di routing intra-AS e inter-AS.



- intra-AS routing determina le entrate per le destinazioni dentro l'AS
- inter-AS & intra-AS determina le entrate per destinazioni esterne

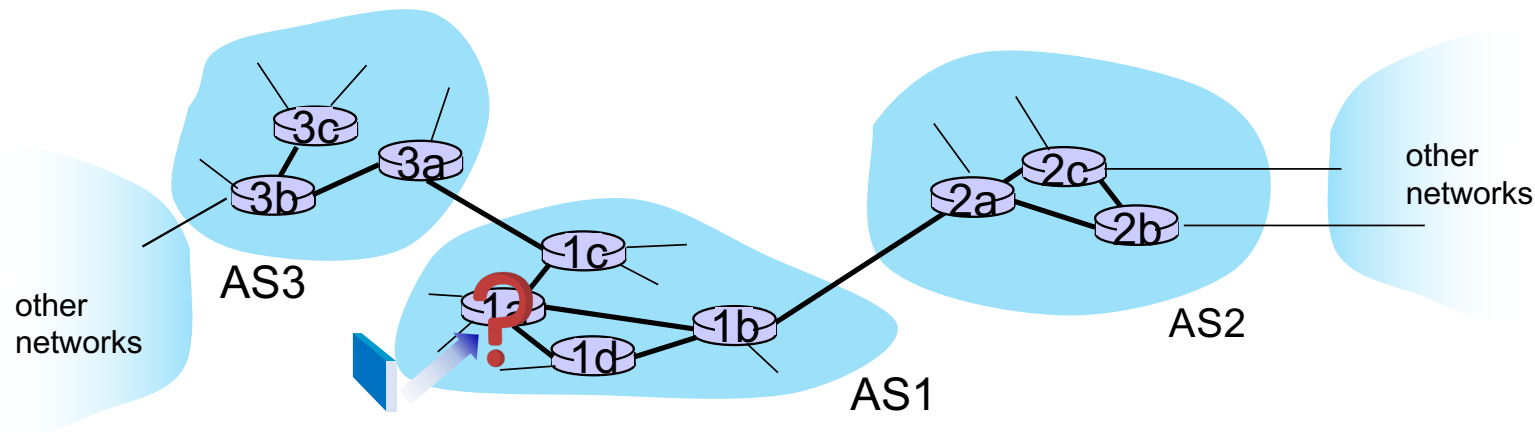
Inter-AS routing: il ruolo nell'intra-domain forwarding

- Supponiamo che il router in AS1 riceva un datagram con destinazione esterna a AS1:

? • Il router dovrebbe inoltrare il datagramma al gateway router in AS1, ma quale?

Routing inter-domain in AS1 deve:

1. Capire quali sono le destinazioni che si possono raggiungere attraverso AS2, quali attraverso AS3
2. Propagare queste informazioni di raggiungibilità per tutti i routers in AS1



Intra-AS routing: routing all'interno dell'AS

Protocolli di routing intra-AS più comuni:

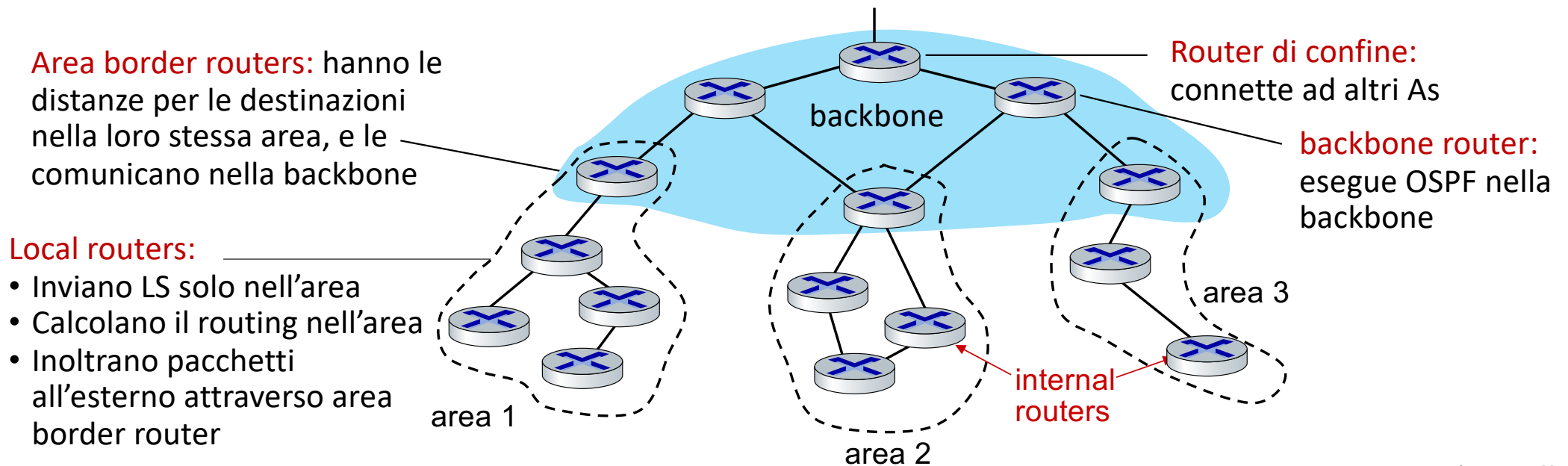
- **RIP: Routing Information Protocol** [RFC 1723]
 - classico DV: DVs scambiati ogni 30 secs
 - Non è più usato
- **EIGRP: Enhanced Interior Gateway Routing Protocol**
 - Basato su DV
 - È stato proprietario di Cisco per decenni (diventò open nel 2013 [RFC 7868])
- **OSPF: Open Shortest Path First** [RFC 2328]
 - link-state routing
 - È lo stesso di IS-IS protocol (ISO standard, non standard RFC)

OSPF (Open Shortest Path First) routing

- “open”: pubblicamente disponibile
- link-state classico
 - ogni router invia a tutti degli OSPF link-state advertisements (direttamente su IP, invece che usare TCP/UDP) a tutti gli altri routers nell'AS
 - Sono possibili diverse metriche per il costo del link: bandwidth, delay
 - Ogni router conosce tutta la topologia, usa l'algoritmo di Dijkstra per calcolare la tabella di forwarding
- *security*: tutti i messaggi OSPF sono autenticati (pre prevenire intrusion malevole)

OSPF gerarchico

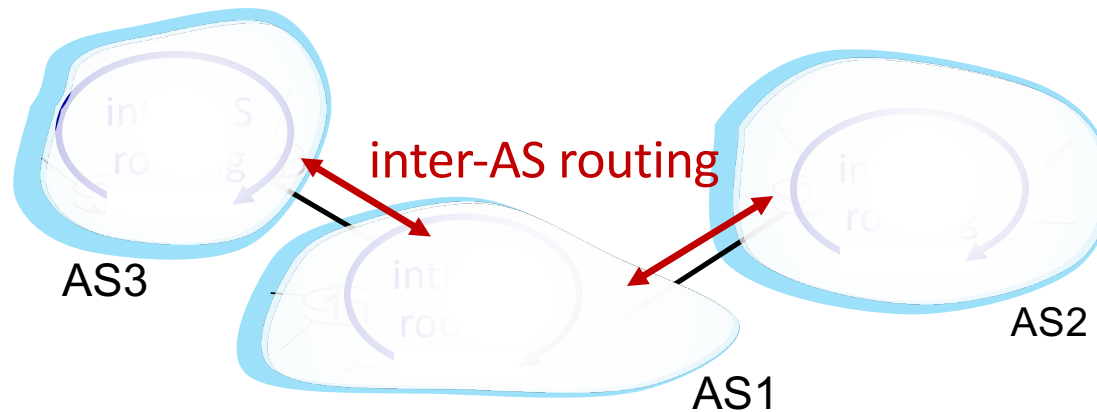
- Gerarchia a due livelli: local area, backbone.
 - Link-state advertisements inviati solo nella local area, or backbone
 - Ogni nodo ha la topologia dettagliata per l'area; per altre destinazioni sa solo le direzioni



Network layer: roadmap del piano di controllo

- Introduzione
- Protocolli di Routing
 - link state
 - distance vector
 - Intra-ISP routing: OSPF
- **Routing tra ISPs: BGP**

AS Interconnessioni

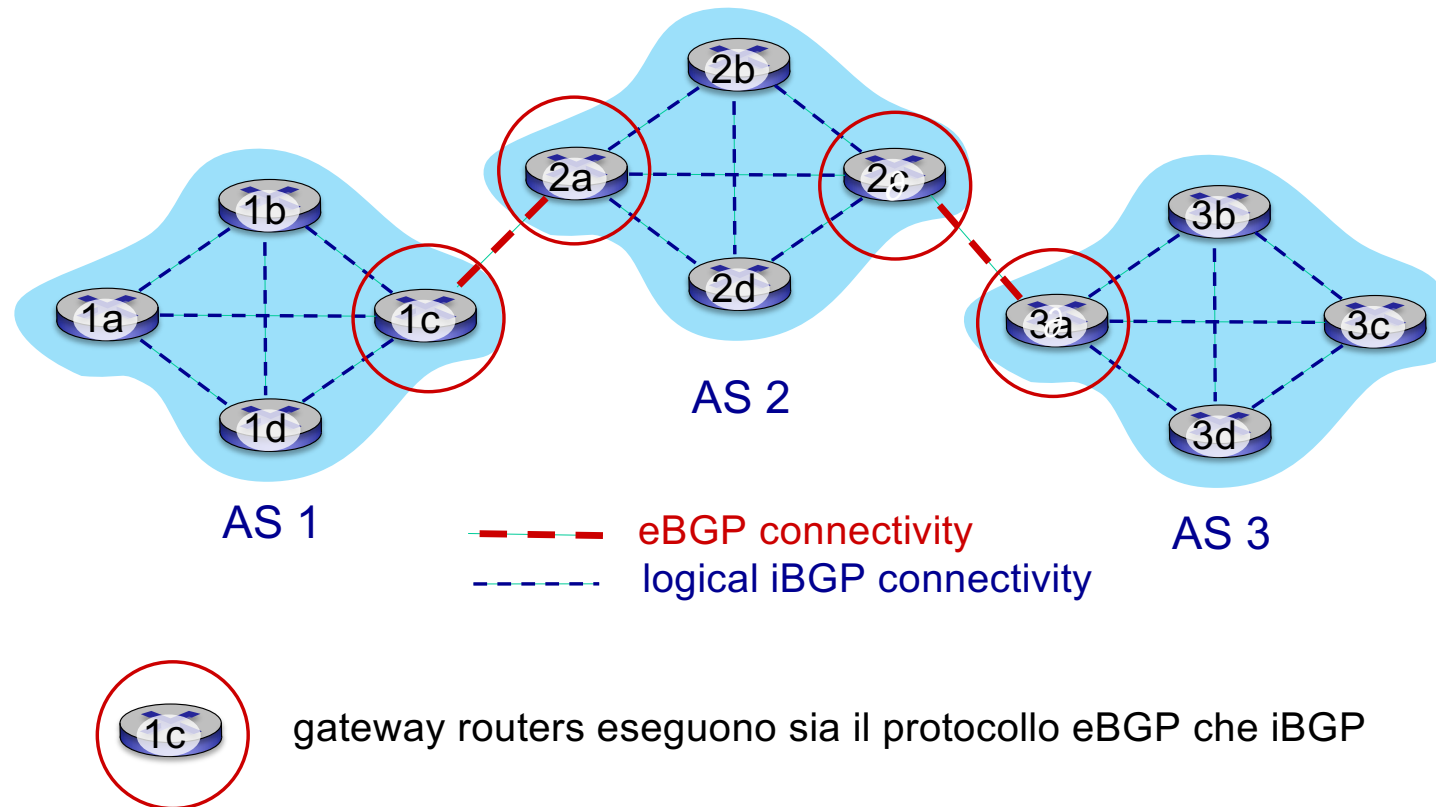


- ✓ **intra-AS (aka “intra-domain”)**: routing tra routers *nello stesso AS* (“*network*”)
- ➡ **inter-AS (aka “inter-domain”)**: routing *tra* AS

Inter-AS routing: BGP

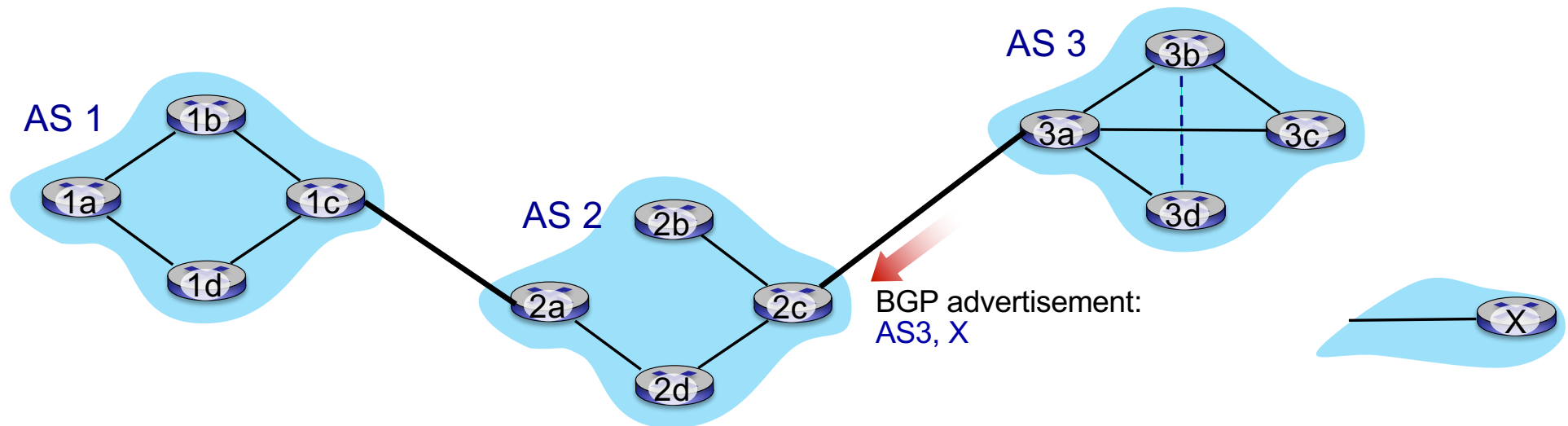
- **BGP (Border Gateway Protocol)**: standard de facto dei protocolli di routing inter-domain
 - “colla che tiene insieme l’Internet”
 - <https://bgp.he.net/country/IT>
- Consente ad una sottorete di pubblicizzare la sua esistenza al resto di Internet: *“sono qui, ecco chi posso raggiungere e come”*
- BGP fornisce ad ogni AS un mezzo per:
 - Ottenere informazioni sulla raggiungibilità delle reti di destinazione di AS vicini (**eBGP**)
 - Determinare le rotte per altre reti basandosi su informazioni di raggiungibilità e *policy*
 - Propagare informazioni di raggiungibilità a tutti i router nell’AS (**iBGP**)
 - **Publicizzare** (alle reti vicine) le informazioni di raggiungibilità delle destinazioni

Connessioni interne e esterne, eBGP, iBGP



BGP – Le basi

- **BGP session:** due router BGP si scambiano messaggi BGP su connessioni TCP semi-permanenti:
 - pubblicizzare *paths* verso diverse sottoreti (prefissi) di destinazione (BGP è un protocollo “path vector”)
- Quando il gateway di AS3 3a pubblicizza *path AS3,X* al gateway di AS2, 2c:
 - AS3 *promette* a AS2 che inoltrerà datagrammi verso X



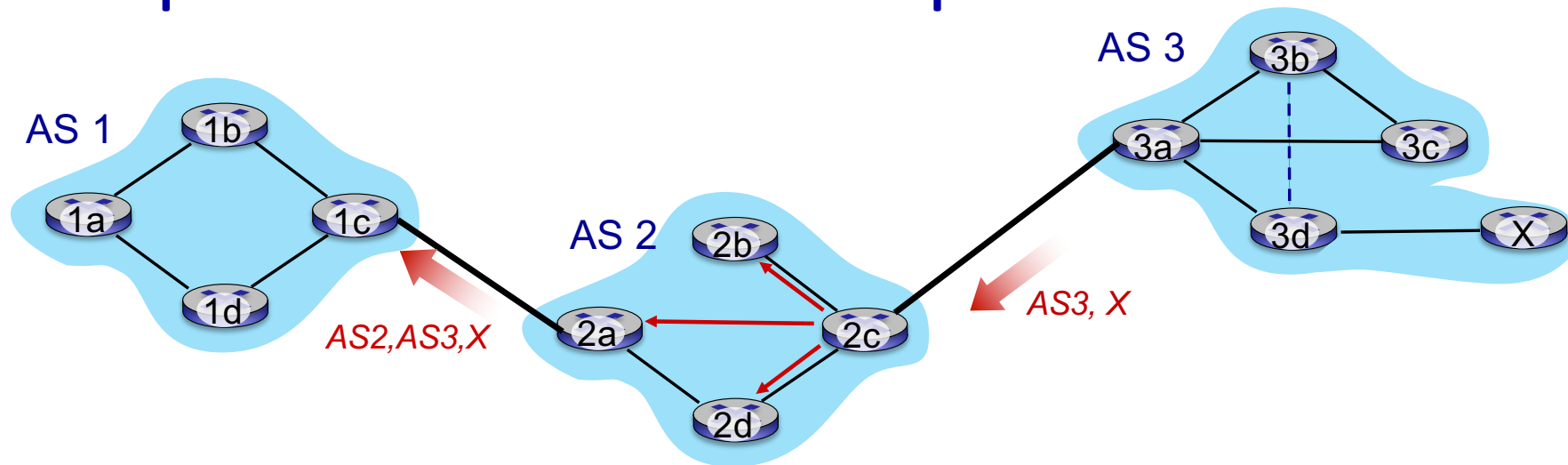
BGP – messaggi del protocollo

- Messaggi BGP scambiati tra pari su connessione TCP
- Messaggi BGP [RFC 4371]:
 - **OPEN**: apre una connessione TCP con il peer BGP remoto e autentica il peer BGP mittente
 - **UPDATE**: pubblicizza un nuovo path (o ne rimuove uno vecchio)
 - **KEEPALIVE**: tiene la connessione viva in assenza di UPDATES; manda anche l'ACK alla OPEN request
 - **NOTIFICATION**: riporta errori nei messaggi precedenti; usato anche per chiudere le connessioni

Attributi dei percorsi e rotte BGP

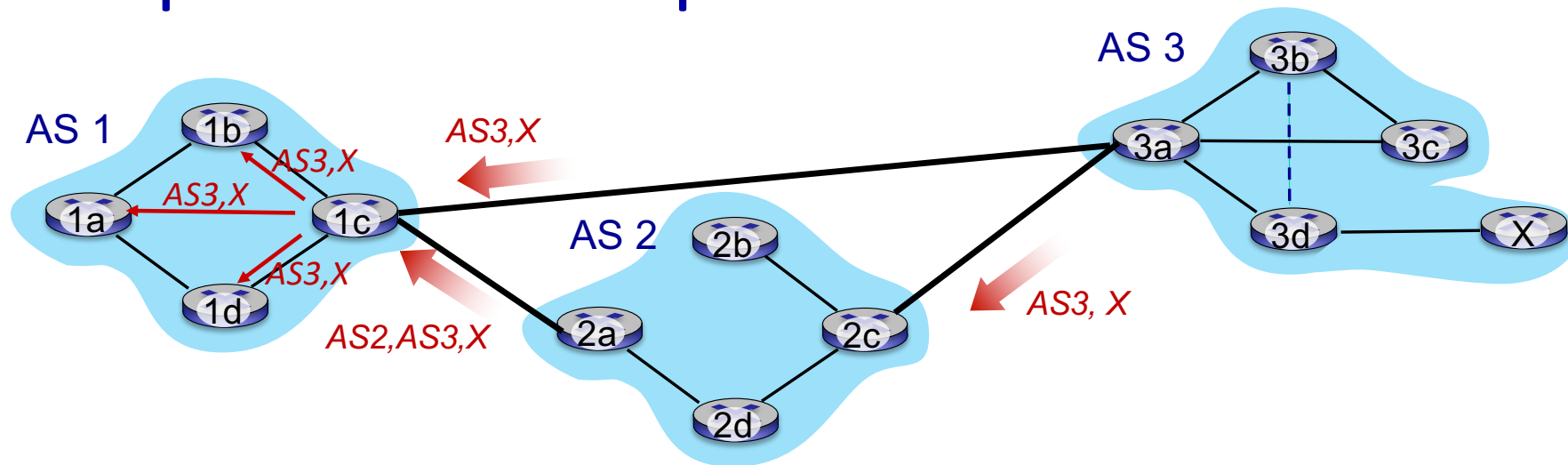
- BGP pubblicizza una rotta come: prefisso sottorete + attributi
 - Prefisso sottorete: destinazione pubblicizzata
 - Due attribute importanti:
 - **AS-PATH**: lista degli AS per i quali è passata la pubblicizzazione del prefisso
 - **NEXT-HOP**: indica l'indirizzo di un'interfaccia del router di confine in un AS per andare al next-hop AS
- **Routing basato su policy:**
 - Il gateway che riceve la pubblicizzazione di una rotta usa *import policy* per accettare/declinare la rotta (e.g., non mandare pacchetti passando da AS Y).
 - La policy dell'AS determina anche quando *pubblicizzare* un percorso agli AS vicini

BGP pubblicizzazione dei percorsi



- Il router 2c di AS2 riceve rotta **AS3,X** (via eBGP) dal router 3a di AS3
- In base alle policy di AS2, il router 2c accetta il percorso AS3,X, e propaga l'informazione (via iBGP) a tutti i router di AS2
- In base alle policy di AS2, il router 2a pubblicizza (via eBGP) il percorso **AS2, AS3, X** al router 1c di AS1

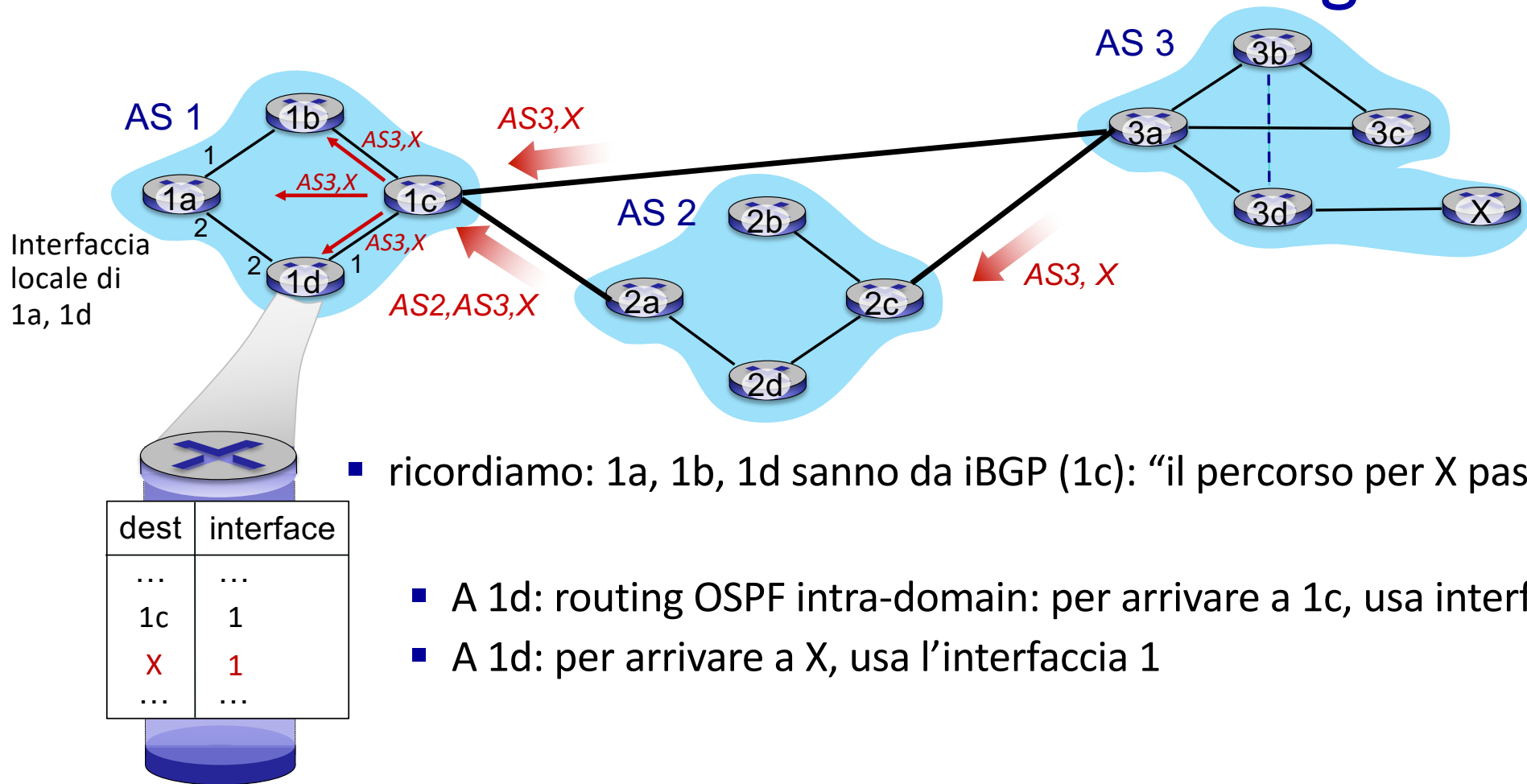
BGP percorsi multipli



Il router di confine (gateway) può conoscere percorsi **multipli** per destinazione:

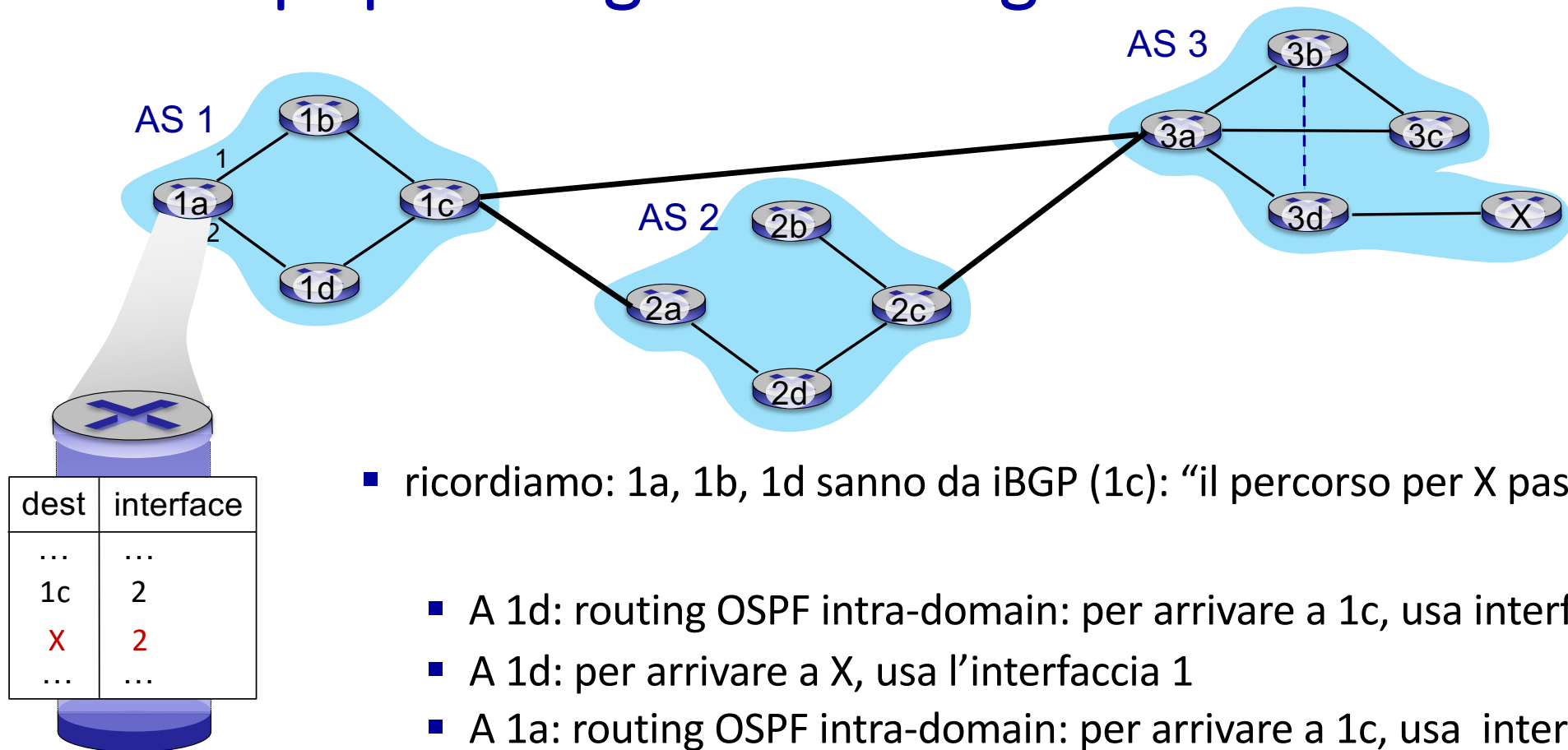
- Il router 1c di AS1 conosce il percorso **AS2, AS3, X** da 2a
- Il router 1c di AS1 conosce il percorso **AS3, X** da 3a
- Basandosi su una **policy**, 1c sceglie il percorso **AS3, X** e lo pubblicizza in AS1 via iBGP

BGP: costruire le tabelle di forwarding



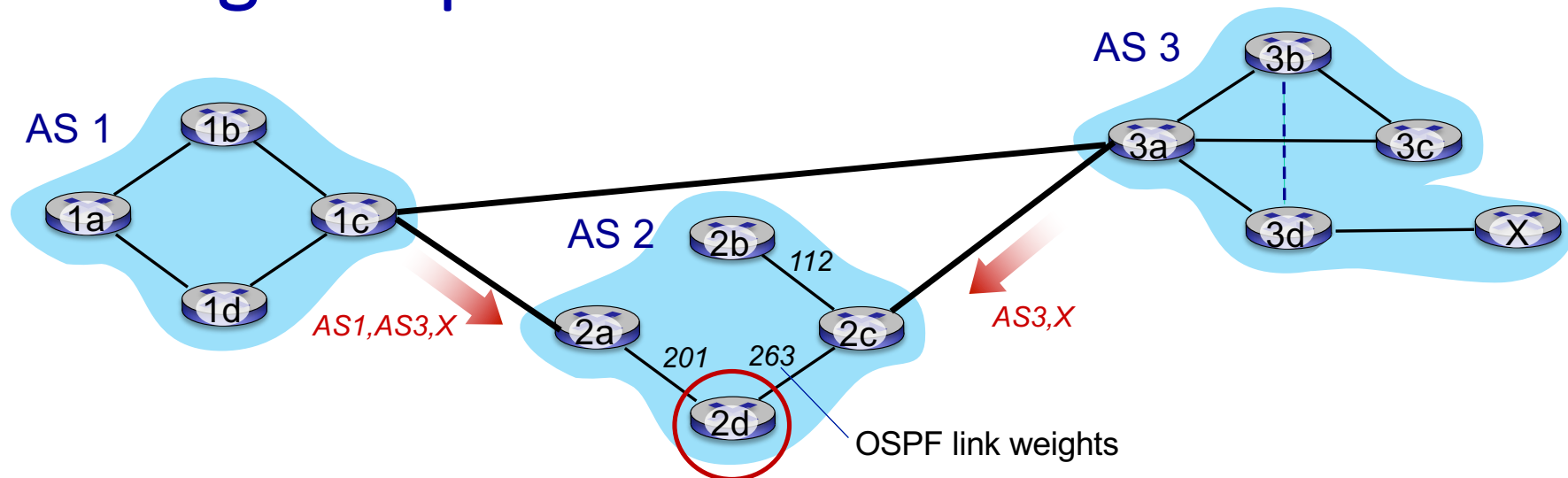
- ricordiamo: 1a, 1b, 1d sanno da iBGP (1c): “il percorso per X passa da 1c”
- A 1d: routing OSPF intra-domain: per arrivare a 1c, usa interfaccia 1
- A 1d: per arrivare a X, usa l’interfaccia 1

BGP: populating forwarding tables



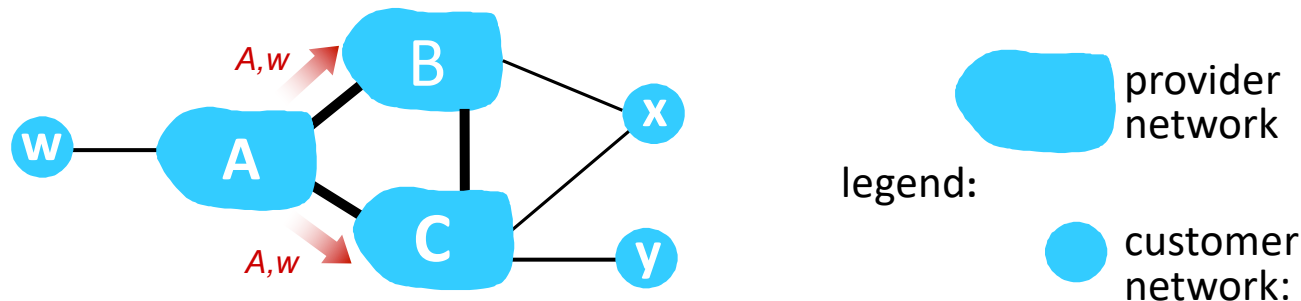
- ricordiamo: 1a, 1b, 1d sanno da iBGP (1c): “il percorso per X passa da 1c”
 - A 1d: routing OSPF intra-domain: per arrivare a 1c, usa interfaccia 1
 - A 1d: per arrivare a X, usa l’interfaccia 1
 - A 1a: routing OSPF intra-domain: per arrivare a 1c, usa interfaccia 2
 - A 1a: per arrivare a X, usa interfaccia 2

Routing Hot potato



- 2d sa (via iBGP) che può arrivare a X via 2a o 2c
- **Routing hot potato:** scegliere il gateway locale che ha il costo *intra-domain* minore (e.g., 2d sceglie 2a, anche se si devono passare più AS per arrivare a X): non si preoccupa dei costi inter-domain!

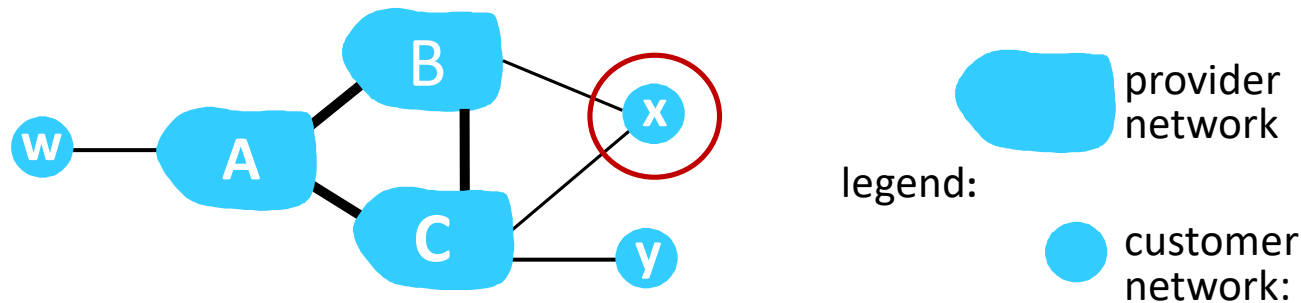
BGP: soddisfare la policy con pubblicizzazione



Un ISP vuole solo inoltrare il traffico verso/da le reti dei suoi client (non vuole inoltrare traffic diretto ad altri ISP – tipica “real world” policy)

- A pubblicizza il percorso Aw a B e C
- B *sceglie di non pubblicizzare* BA_w a C!
 - B non avrebbe nessun ritorno a inoltrare CBA_w, perchè nè C, A, w sono suoi clienti
 - C non conosce il percorso CBA_w
- C inoltra CA_w (non usando B) per arrivare a w

BGP: soddisfare la policy con pubblicizzazione



Un ISP vuole solo inoltrare il traffico verso/da le reti dei suoi client (non vuole inoltrare traffic diretto ad altri ISP – tipica “real world” policy)

- A,B,C sono reti dei **provider**
- x,w,y sono **clienti** (delle reti dei provider)
- x è **dual-homed**: collegato a due reti
- **policy**: x non vuole inoltrare da B a C via x
 - .. x non pubblicizzerà a B la rotta per C

BGP selezione delle rotte

- I router possono conoscere più di una rotta per AS di destinazione, le rotte vengono selezionate sulla base di:
 1. Preferenze locali date dai valori di alcuni attributi : policy
 2. AS-PATH più corto
 3. NEXT-HOP router più vicino: routing hot potato
 4. Altro

Perchè il routing Intra-, Inter-AS è diverso ?

Policy:

- inter-AS: amministratori vogliono il controllo su come il traffico viene instradato, su chi instrada attraverso la sua rete
- intra-AS: amministratore singolo, la policy non è un grande problema

scala:

- Routing gerarchico riduce la dimensione delle tabelle, riduce gli update da mandare

prestazioni:

- intra-AS: si può concentrare sulle prestazioni
- inter-AS: la policy è più importante delle prestazioni